Machine Learning
The Working Text

The mathematics, the code, and the discipline —
from first principles to professional capability

Built from **Mathematics of Machine Learning** (Danka) · **Prompting Scikit-Learn for Machine Learning**
PyTorch: The Practical Guide (Gollnick) · **Time Series with PyTorch** (Davidson & Ma)
Google Python Style Guide · the 30 foundational papers in `papers/01..30`

Every derivation is worked. Every code listing runs.
Every claim carries the page it came from.

Contents

How to use this text

- The sources, and what each one is for
- Notation
- The environment

Part I — The Mathematical Engine

1 · Vectors, spaces, and the array

- 1.1 What a vector space actually buys you
- 1.2 Span, independence, basis
- 1.3 Data as a matrix
- 1.4 The array is not a list
- 1.5 Gate for this chapter

2 · Geometry: norms, angles, orthogonality

- 2.1 Norms
- 2.2 Inner products and angle
- 2.3 Projection, derived
- 2.4 Gram–Schmidt
- 2.5 Gate for this chapter

3 · Matrices as linear maps

- 3.1 The map is the point
- 3.2 The four quantities
- 3.3 Transformations you can see
- 3.4 The determinant
- 3.5 Gate for this chapter

4 · Solving linear systems

- 4.1 Elimination
- 4.2 LU
- 4.3 Never invert
- 4.4 Least squares, derived
- 4.5 Gate for this chapter

5 · Eigenvalues, the spectral theorem, and the SVD

- 5.1 Eigenvectors
- 5.2 The spectral theorem
- 5.3 The singular value decomposition
- 5.4 Low rank, and why it is the whole game
- 5.5 PCA, derived three ways
- 5.6 Gate for this chapter

6 · Derivatives and optimisation in one variable

6.1 Three readings of one number
6.2 Differentiating numerically
6.3 Gradient descent
6.4 Gate for this chapter

7 · Many variables: gradients, Jacobians, backpropagation
7.1 Gradient
7.2 Jacobian and the chain rule
7.3 Backpropagation, derived in full
7.4 Curvature and convexity
7.5 The curse of dimensionality
7.6 Gate for this chapter

8 · Probability, information, and where losses come from
8.1 The axioms
8.2 Random variables and distributions
8.3 Expectation, variance, and the law of large numbers
8.4 Information: entropy, cross-entropy, KL
8.5 Maximum likelihood: where every loss comes from
8.6 Gate for this chapter

Part II — Learning from Data

9 · Framing the problem
9.1 Decide the metric before the first fit
9.2 Problem types
9.3 Schema before model

10 · Leakage
10.1 The demonstration
10.2 The taxonomy
10.3 The checklist
10.4 The one defence that scales

11 · Pipelines that cannot leak
11.1 The shape
11.2 Prove it
11.3 Splitters

12 · Linear models
12.1 The bias–variance decomposition
12.2 Ridge, derived — and what α really does
12.3 L1 versus L2, finished properly
12.4 Logistic regression

13 · Trees and ensembles
13.1 One tree, one split
13.2 Bagging and boosting

14 · Evaluation

- 14.1 The confusion matrix, worked
- 14.2 Regression metrics
- 14.3 Cross-validation and the optimism of selection
- 14.4 Every number carries an interval
- 14.5 Calibration

15 · Unsupervised structure

- 15.1 PCA in practice
- 15.2 k-means, and why it always converges
- 15.3 DBSCAN
- 15.4 How to know if a clustering is real

Part II gate

Part III — Neural Networks in PyTorch

16 · Tensors and automatic differentiation

- 16.1 A tensor is an array that remembers
- 16.2 What backward() actually computes
- 16.3 Prove autograd to yourself once

17 · Layers, activations, and losses

- 17.1 Activations and their derivatives
- 17.2 Softmax and its Jacobian
- 17.3 Losses are likelihoods
- 17.4 Initialisation, derived
- 17.5 Normalisation

18 · The training loop

- 18.1 Dataset and DataLoader
- 18.2 Batch size
- 18.3 Optimisers
- 18.4 Determinism and checkpoints
- 18.5 Reading a loss curve

19 · Convolution

- 19.1 Why not a dense layer
- 19.2 The arithmetic you must know
- 19.3 Convolution is a matrix multiply

20 · Autoencoders and the variational autoencoder

- 20.1 Autoencoders
- 20.2 The VAE, derived

21 · Attention and transformers

- 21.1 Scaled dot-product attention
- 21.2 The rest of the block

Part III gate

Part IV — Sequences and Uncertainty

22 · The structure of temporal data

- 22.1 What makes it different
- 22.2 Decomposition
- 22.3 ACF and PACF
- 22.4 Stationarity
- 22.5 Structure determines the splitter

23 · Honest temporal evaluation

- 23.1 Never shuffle
- 23.2 Metrics, and the two traps
- 23.3 Baselines you must beat
- 23.4 Horizon

24 · Neural forecasting

- 24.1 Windowing
- 24.2 Recurrent networks
- 24.3 Transformers for forecasting

25 · Uncertainty: conformal prediction

- 25.1 Why a point forecast is not enough
- 25.2 The mechanism
- 25.3 Adaptive intervals
- 25.4 The honest caveats
- 25.5 Anomaly detection, briefly

Part IV gate

Part V — Professional Practice

26 · The code contract

- 26.1 Conventions the style guide does not cover

27 · The method

- 27.1 Four roles, never two at once
- 27.2 Thresholds before code
- 27.3 Status tags
- 27.4 The audit pack

28 · From model to system

- 28.1 Tracking
- 28.2 Serving
- 28.3 Explainability
- 28.4 Governance

29 · The route through

- 29.1 Twenty-four weeks
- 29.2 The daily shape
- 29.3 The capstone

Part VI — Four Worked Projects

30 · Project 1 — Tabular classification

- 30.1 The data is the lesson
- 30.2 The threshold is a business parameter
- 30.3 Nested CV, and reading the gap
- 30.4 Where the gate was wrong

31 · Project 2 — A convolutional classifier, trained by hand

- 31.1 Check the gradients before you trust the training
- 31.2 Measure the learning rate; do not guess it
- 31.3 Where the gate was wrong, again

32 · Project 3 — Forecasting with real intervals

- 32.1 Diagnostics first
- 32.2 Baselines, and what beating them means
- 32.3 The headline: coverage fails exactly where it must

33 · Project 4 — A VAE with the ELBO taken apart

- 33.1 The beta sweep
- 33.2 Where the gate was wrong, a third time
- 33.3 The fair baseline

34 · Project 5 — Serving a model, and watching it afterwards

- 34.1 Ship the pipeline, not the recipe
- 34.2 Reject at the boundary
- 34.3 Latency, and where it actually goes
- 34.4 Watching the inputs, because the labels are late
- 34.5 Rollback is a feature you test
- 34.6 What this project does not cover

35 · What the five have in common

Appendix A · What these sources do not cover

Appendix B · The exercises

Appendix C · The worked projects

Appendix D · Working the exercises in Colab

- The shape of a notebook
- Why the notebooks are tested
- The projects, too
- Two bugs the validators caught

How to use this text

This is not a summary of four books. It is the material itself — the derivations, the algorithms, the code, and the judgement — written out so you can learn from this page rather than being sent somewhere else. Where a source goes deeper than a working text should, you will find a marker like *[MML p.207]*, telling you exactly which page to open and why.

Read it with your hands. Every chapter ends in code that runs. Type it; do not paste it. The difference between a reader and a practitioner is measured in keystrokes. Part VI then puts the pieces together into four complete systems you can run in a minute, in `ml-mastery/projects/`.

Three rules hold this text together.

1. *Implement before you use.* Everything in Part I exists in NumPy already. You write it once yourself, then you use the library version forever with an informed conscience.
2. *One number, one source.* Metrics are defined once, in code, and every comparison in the book calls that one implementation. Three of the source books define overlapping metrics that disagree at the edges; those disagreements are resolved here, with the edge cases as tests.
3. *A result without a gate is not a result.* Each chapter states what must be numerically true for you to move on. Not "I understood it" — a number, within a tolerance, that you can show somebody.

The sources, and what each one is for

Tag	Source	What it is best at
[MML]	*Mathematics of Machine Learning*, Tivadar Danka, Packt 2025	The mathematics, with proofs and intuition that actually connect. Printed page numbers.
[SKL]	*Prompting Scikit-Learn for Machine Learning*	Leak-safe pipelines, evaluation discipline, the workflow around a model. Chapter + section anchors.
[PTG]	*PyTorch: The Practical Guide*, Bert Gollnick, Rheinwerk 2026	PyTorch mechanics end to end, through to deployment. Printed page numbers.
[TSP]	*Time Series with PyTorch*, Davidson & Ma, Packt 2026	Temporal data, honest evaluation, uncertainty. PDF page indices.
[GPSG]	*Google Python Style Guide*	How the code should look when somebody else reads it.
[30u30]	`papers/01..30` in this repository	Thirty foundational papers, implemented.

Notation

Symbol	Meaning
x, y	scalars
$\mathbf{x}$, $\mathbf{y}$	column vectors; x_i is the i-th entry
A, B	matrices; A^T is the transpose; a_{ij} the entry in row i, column j
$\mathbb{R}^n$	the space of real n-tuples
$\|\mathbf{x}\|_2$	Euclidean norm, $\sqrt{(\Sigma\, x_i^2)}$
$\langle \mathbf{x}, \mathbf{y} \rangle$	inner product, $\Sigma\, x_i y_i$
∇f	gradient of f, the vector of partial derivatives
h	learning rate (the step size in gradient descent), following [MML p.383]
$\mathbb{E}[X]$	expected value of the random variable X
p, q	probability distributions
θ	a model's parameters, collectively
$L(\theta)$	the loss as a function of parameters

Code follows the Google Python Style Guide: 80 columns, four-space indents, type annotations on public functions, Google-style docstrings. Inside a derivation, short mathematical names (`A`, `x`, `lr`) are used deliberately — the style guide permits local names that match the mathematics *[GPSG §3.16.5]*.

The environment

Everything here runs on NumPy; Parts II–IV add scikit-learn, PyTorch and statsmodels. The testbed that accompanies this text brings all of it up with one command:

```
cd ml-mastery\testbed
.\setup.ps1
```

If a package is missing, the experiments say so and mark the affected result `UNVERIFIED` rather than skipping it quietly. That behaviour is the whole philosophy of this text in miniature: **the absence of a check is itself reported.**

Part I — The Mathematical Engine

Machine learning is three pieces of mathematics wearing a trench coat: linear algebra moves the data, calculus moves the parameters, probability tells you what any of it means. Part I builds all three from the ground, in code.

The rule for this part is **implement before you use**. Everything here exists in NumPy. Writing it once yourself is what converts a formula you have seen into a tool you own — and NumPy then serves as the oracle that tells you whether you got it right.

---

1 · Vectors, spaces, and the array

1.1 What a vector space actually buys you

A **vector space** over $\mathbb{R}$ is a set V with addition and scalar multiplication that behave the way you already expect: addition is commutative and associative, there is a zero, every element has a negative, and scalar multiplication distributes over both kinds of addition *[MML p.9]*.

That list of axioms looks like bookkeeping. It is not. It is a *licence*. Once you know a set is a vector space, every theorem about vector spaces applies to it — and the set does not have to look like arrows. These are all vector spaces:

- $\mathbb{R}^n$, the n-tuples of real numbers — a row of a dataset;
- the real-valued functions on an interval — a signal, a density;
- the polynomials of degree $\le k$ — a feature expansion;
- the $m \times n$ matrices — images, weight layers.

This is why a single body of theory covers tabular rows, images and distributions at once. When you later read that "the activations live in a 512-dimensional space," that sentence is doing real work: it means every tool in this part applies to them.

1.2 Span, independence, basis

A **linear combination** of $\mathbf{v}_1, \dots, \mathbf{v}_k$ is any $c_1\mathbf{v}_1 + \cdots + c_k\mathbf{v}_k$. The set of all of them is the **span** *[MML p.16]*.

The vectors are **linearly independent** when the only combination equal to zero is the one with every $c_i = 0$ *[MML p.15]*. Equivalently — and this is the form you will use — none of them is reachable from the others, so none is redundant.

A **basis** is a linearly independent set that spans the whole space: the minimal description with nothing left over *[MML p.18]*. The number of basis vectors is the **dimension**.

Worked example — uniqueness of coordinates

Why does independence matter? Because it is exactly what makes coordinates *unique*, and uniqueness is what lets you speak of "the" representation of a data point at all.

Suppose a vector $\mathbf{x}$ had two representations in the same basis:

$$\mathbf{x} = \Sigma_i\, a_i\mathbf{v}_i = \Sigma_i\, b_i\mathbf{v}_i$$

Subtract them: $\Sigma_i(a_i - b_i)\mathbf{v}_i = \mathbf{0}$. By independence every coefficient of that combination must be zero, so $a_i = b_i$ for all i. The two representations were the same one. ■

Lose independence and the proof collapses — and with it, the meaning of a coefficient. Take $\mathbf{v}_1 = (1,0)$, $\mathbf{v}_2 = (0,1)$, $\mathbf{v}_3 = (1,1)$, and the vector $\mathbf{x} = (2,3)$:

$$\mathbf{x} = 2\mathbf{v}_1 + 3\mathbf{v}_2 + 0\mathbf{v}_3 = 1\mathbf{v}_1 + 2\mathbf{v}_2 + 1\mathbf{v}_3 = 0\mathbf{v}_1 + 1\mathbf{v}_2 + 2\mathbf{v}_3$$

Three different "explanations" of the same point, all equally valid. **This is collinearity in a regression, stated exactly.** The predictions are identical; the coefficients are arbitrary; and any story you tell about "the effect of feature 3" is a story about which of infinitely many solutions your solver happened to land on.

> **Practitioner's rule.** `np.linalg.matrix_rank(X) < X.shape[1]` means redundancy, and it means your coefficients are not identifiable. Ridge (Chapter 12) does not merely shrink them — it *picks one*, by adding a criterion the data could not supply.

1.3 Data as a matrix

Stack n observations of d features and you have the **design matrix** X ∈ ℝ^(n×d) *[MML p.110]*. Two readings of the same object, both useful:

- **by rows** — n points in a d-dimensional feature space (the geometric view: clustering, distances, neighbourhoods);
- **by columns** — d vectors in an n-dimensional sample space (the algebraic view: collinearity, rank, least squares).

Most confusion about a model evaporates when you decide, deliberately, which of these two pictures you are in.

1.4 The array is not a list

NumPy's `ndarray` is a typed, fixed-size, contiguous block with a shape. Python's `list` is a box of pointers to arbitrary objects. That difference is the whole performance story *[MML p.29, p.35]*.

A Python loop over a list pays, per element, for pointer chasing, type dispatch and the construction of a new boxed float. NumPy pays once for the whole array and runs a tight typed loop in C. The asymptotics are identical — the constant is one to two orders of magnitude *[MML p.37]*.

```
"""Measures the cost of a Python loop against a vectorised dot product."""

import time

import numpy as np


def dot_python(a: list, b: list) -> float:
    """Computes a dot product with a Python loop."""
    total = 0.0
    for x, y in zip(a, b):
        total += x * y
    return total


def benchmark(n: int) -> tuple:
    """Times both implementations at length n.

    Args:
        n: Vector length.

    Returns:
        A tuple of (python_seconds, numpy_seconds, speedup).
    """
    rng = np.random.default_rng(0)
    arr_a, arr_b = rng.normal(size=n), rng.normal(size=n)
    list_a, list_b = arr_a.tolist(), arr_b.tolist()

    start = time.perf_counter()
    slow = dot_python(list_a, list_b)
    python_s = time.perf_counter() - start

    start = time.perf_counter()
    fast = float(np.dot(arr_a, arr_b))
    numpy_s = time.perf_counter() - start

    assert abs(slow - fast) < 1e-8 * max(1.0, abs(fast))
    return python_s, numpy_s, python_s / max(numpy_s, 1e-12)


for size in (10_000, 100_000, 1_000_000):
    py_s, np_s, ratio = benchmark(size)
    print(f'n={size:>9,}  python {py_s:7.4f}s  numpy {np_s:7.5f}s  '
          f'speedup {ratio:6.0f}x')
```

Run it. Do not take the number on faith and do not take mine — the ratio depends on your BLAS, your cache and your CPU, and no source in this stack publishes a figure you could check against. Record what *your* machine does.

Broadcasting is the other half of thinking in arrays. When shapes differ, NumPy compares them from the right and stretches any axis of length 1:

```
x = np.arange(12).reshape(4, 3)       # (4, 3)
column_means = x.mean(axis=0)         # (3,)
centred = x - column_means            # (4,3) - (3,) -> (4,3)
```

That last line is feature centring, the first step of PCA, written without a loop. The mental discipline: *never write a loop over samples.*

A worked trap: the silent broadcast

Broadcasting is also the most common silent bug in array code. Consider computing residuals where `y` has shape (n,) and `y_pred` has shape (n, 1):

$$(n,) - (n,1) \to (n,n)$$

NumPy aligns from the right, sees (n,) as (1,n), and produces an n×n matrix of *every pairwise difference*. Your RMSE then averages n^2 numbers, most of them meaningless, and returns a plausible-looking float. Nothing raises.

```
y = np.array([1.0, 2.0, 3.0])
y_pred = np.array([[1.1], [1.9], [3.2]])
print((y - y_pred).shape)              # (3, 3)  -- not (3,)
print(float(np.sqrt(np.mean((y - y_pred) ** 2))))   # 0.99..., nonsense
print(float(np.sqrt(np.mean((y - y_pred.ravel()) ** 2))))   # 0.14, correct
```

This is why every metric in this text validates shapes and refuses mismatches rather than broadcasting them.

1.5 Gate for this chapter

Check	Threshold
`matrix_rank` detects a deliberately collinear column	rank = d − 1
Your vectorised dot matches the loop	within 1e-8 relative
You can state the speedup on your machine	a number, recorded
You can reproduce the three-representation example	exact

2 · Geometry: norms, angles, orthogonality

2.1 Norms

A **norm** assigns a length $\|\mathbf{x}\|$ with three properties: it is zero only for the zero vector, it scales ($\|c\mathbf{x}\| = |c|\,\|\mathbf{x}\|$), and it satisfies the triangle inequality *[MML p.49]*. The family you will meet:

$$\|\mathbf{x}\|_1 = \Sigma_i\,|x_i| \quad \cdot \quad \|\mathbf{x}\|_2 = \sqrt{(\Sigma_i\, x_i^2)} \quad \cdot \quad \|\mathbf{x}\|_\infty = \max_i\,|x_i|$$

Worked example — the same vector, three lengths

Take $\mathbf{x} = (3, -4, 12)$:

Norm	Computation	Value
$\|\mathbf{x}\|_1$	3 + 4 + 12	**19**
$\|\mathbf{x}\|_2$	$\sqrt{(9 + 16 + 144)} = \sqrt{169}$	**13**
$\|\mathbf{x}\|_\infty$	max(3, 4, 12)	**12**

Three honest answers to "how big is this?", and they are ordered $\|x\|_\infty \le \|x\|_2 \le \|x\|_1$ — which holds in general and is worth knowing, because it tells you an L1 penalty is always at least as aggressive as an L2 one at the same coefficient size.

The unit balls differ in a way that has consequences you will meet in Chapter 12. The L2 ball is round. The L1 ball is a diamond with **corners on the axes** — and a corner on an axis is a point where some coordinates are exactly zero. That geometric fact is the whole explanation for L1's sparsity, and we will finish the argument when we have the loss contours.

Every norm induces a **distance**, $d(\mathbf{x}, \mathbf{y}) = \|\mathbf{x} - \mathbf{y}\|$ *[MML p.55]*, which is where k-nearest-neighbours, k-means and every clustering algorithm get their notion of "near".

2.2 Inner products and angle

The **inner product** $\langle \mathbf{x}, \mathbf{y} \rangle = \Sigma_i\, x_i y_i = \mathbf{x}^\mathsf{T}\mathbf{y}$ generates the Euclidean norm, $\|\mathbf{x}\|_2 = \sqrt{\langle \mathbf{x}, \mathbf{x} \rangle}$ *[MML p.58]*, and with it the angle between vectors *[MML p.61]*:

$$\cos\theta = \langle x, y \rangle \,/\, (\|x\|_2\,\|y\|_2)$$

Why that formula is the cosine

This is usually asserted. It follows from the law of cosines in one step, and the derivation is worth seeing because it explains why dot products measure *alignment*.

The law of cosines on the triangle with sides $\mathbf{x}$, $\mathbf{y}$ and $\mathbf{x} - \mathbf{y}$:

$$\|x - y\|^2 = \|x\|^2 + \|y\|^2 - 2\|x\|\|y\|\cos\theta$$

Expand the left side algebraically instead:

$$\|x - y\|^2 = \langle x - y, x - y\rangle = \|x\|^2 - 2\langle x, y\rangle + \|y\|^2$$

Set the two right-hand sides equal; $\|\mathbf{x}\|^2$ and $\|\mathbf{y}\|^2$ cancel, leaving $-2\langle\mathbf{x},\mathbf{y}\rangle = -2\|\mathbf{x}\|\|\mathbf{y}\|\cos\theta$, hence the formula. ■

So **cosine similarity is not a metaphor** — it is the actual cosine of the actual angle, and that is why it discards magnitude and keeps direction, which is exactly right when a long document and a short one should count as similar if they are about the same thing.

Orthogonality is $\langle\mathbf{x}, \mathbf{y}\rangle = 0$ *[MML p.60]*: zero correlation in the geometric sense, no shared component, nothing of one inside the other.

2.3 Projection, derived

The projection of $\mathbf{x}$ onto the direction of $\mathbf{v}$ is the piece of $\mathbf{x}$ lying along $\mathbf{v}$. Rather than quote it, derive it: we want the scalar c that makes the residual $\mathbf{x} - c\mathbf{v}$ orthogonal to $\mathbf{v}$.

$$\langle x - cv, v\rangle = 0 \implies \langle x, v\rangle = c\langle v, v\rangle \implies c = \langle x, v\rangle / \langle v, v\rangle$$

$$\text{proj}_v(x) = (\langle x, v\rangle / \langle v, v\rangle)\, v$$

That one-line argument — *choose the coefficient that makes the residual orthogonal* — reappears as the normal equations in Chapter 4, as PCA in Chapter 5, and as the conditional expectation in Chapter 8. It is the single most reused idea in this book.

Worked example — projection by hand

Project $\mathbf{x} = (3, 4)$ onto $\mathbf{v} = (1, 0)$:

$$c = (3\cdot1 + 4\cdot0) / (1\cdot1 + 0\cdot0) = 3 \implies \text{proj} = (3, 0),\ \ \text{residual} = (0, 4)$$

Residual $\perp \mathbf{v}$, as designed. Now the matrix form. The projection *matrix* onto span($\mathbf{v}$) is

$$P = vv^T / \langle v, v\rangle$$

and it has two properties worth checking by hand. **Idempotent**: projecting twice is projecting once, $P^2 = P$, because

$$P^2 = (vv^Tvv^T) / \langle v,v\rangle^2 = v(v^Tv)v^T / \langle v,v\rangle^2 = vv^T / \langle v,v\rangle = P$$

And **symmetric**: $P = P^T$, since $(\mathbf{v}\mathbf{v}^T)^T = \mathbf{v}\mathbf{v}^T$. Those two properties *characterise* orthogonal projections — any symmetric idempotent matrix is one.

When the basis is **orthonormal**, coordinates become free: the i-th coordinate of $\mathbf{x}$ is simply $\langle \mathbf{x}, \mathbf{u}_i \rangle$, with no system to solve *[MML p.65]*. This is why orthogonal bases are hunted so persistently — they turn a linear solve into a dot product.

2.4 Gram–Schmidt

Given independent $\mathbf{v}_1, \ldots, \mathbf{v}_k$, build an orthonormal basis for the same span *[MML p.67, code p.89]*. Take each vector, subtract its projections onto everything already fixed, normalise what remains.

Worked example — three vectors, by hand

Start with $\mathbf{v}_1 = (1, 1, 0)$, $\mathbf{v}_2 = (1, 0, 1)$, $\mathbf{v}_3 = (0, 1, 1)$.

Step 1. $\mathbf{u}_1 = \mathbf{v}_1 / \|\mathbf{v}_1\| = (1, 1, 0)/\sqrt{2}$.

Step 2. Remove from $\mathbf{v}_2$ its component along $\mathbf{u}_1$:

$$\langle v_2, u_1 \rangle = (1 + 0 + 0)/\sqrt{2} = 1/\sqrt{2}$$

$$w_2 = (1, 0, 1) - (1/\sqrt{2})\cdot(1, 1, 0)/\sqrt{2} = (1, 0, 1) - (½, ½, 0) = (½, -½, 1)$$

$\|\mathbf{w}_2\| = \sqrt{(¼ + ¼ + 1)} = \sqrt{(3/2)}$, so $\mathbf{u}_2 = (½, -½, 1)/\sqrt{(3/2)}$.

Step 3. Remove from $\mathbf{v}_3$ its components along both:

$$\langle v_3, u_1 \rangle = 1/\sqrt{2}, \quad \langle v_3, u_2 \rangle = (0 - ½ + 1)/\sqrt{(3/2)} = ½/\sqrt{(3/2)}$$

$$w_3 = (0,1,1) - (½,½,0) - (⅙,-⅙,⅓) = (-⅔, ⅔, ⅔)$$

Check: $\langle \mathbf{w}_3, \mathbf{u}_1 \rangle = (-⅔ + ⅔)/\sqrt{2} = 0$ ✓, and $\langle \mathbf{w}_3, \mathbf{u}_2 \rangle = (-⅓ - ⅓ + ⅔)/\sqrt{(3/2)} = 0$ ✓. Normalise and you have an orthonormal basis.

Do that once with a pencil. The code will never be mysterious again.

```
"""Classical and modified Gram-Schmidt, and the difference between them."""

import numpy as np


def gram_schmidt_classical(a: np.ndarray) -> np.ndarray:
    """Orthonormalises columns by subtracting all projections at once.

    Args:
        a: Matrix of shape (n, k) with linearly independent columns.

    Returns:
        Matrix Q of shape (n, k) with orthonormal columns.

    Raises:
        ValueError: If the columns are not linearly independent.
    """
    n, k = a.shape
    q = np.zeros((n, k), dtype=np.float64)
    for j in range(k):
        v = a[:, j].astype(np.float64).copy()
        # Project onto the already-fixed directions, using the ORIGINAL
        # column each time -- this is what makes it "classical".
        for i in range(j):
            v -= (q[:, i] @ a[:, j]) * q[:, i]
        norm = np.linalg.norm(v)
        if norm < 1e-13:
            raise ValueError(f'column {j} is dependent on the earlier ones')
        q[:, j] = v / norm
    return q


def gram_schmidt_modified(a: np.ndarray) -> np.ndarray:
    """Orthonormalises columns, re-projecting against the running residual.

    Numerically this is the one to use: each projection is computed from
    the partially orthogonalised vector, so rounding error is not allowed
    to accumulate unchecked.

    Args:
        a: Matrix of shape (n, k) with linearly independent columns.

    Returns:
        Matrix Q of shape (n, k) with orthonormal columns.

    Raises:
        ValueError: If the columns are not linearly independent.
    """
    n, k = a.shape
    q = np.zeros((n, k), dtype=np.float64)
    for j in range(k):
        v = a[:, j].astype(np.float64).copy()
        for i in range(j):
            v -= (q[:, i] @ v) * q[:, i]      # note: v, not a[:, j]
        norm = np.linalg.norm(v)
        if norm < 1e-13:
            raise ValueError(f'column {j} is dependent on the earlier ones')
        q[:, j] = v / norm
    return q


def orthogonality_error(q: np.ndarray) -> float:
    """Returns the largest deviation of Q^T Q from the identity."""
    return float(np.abs(q.T @ q - np.eye(q.shape[1])).max())
```

Why the two versions differ

Mathematically they are identical: in exact arithmetic $\langle \mathbf{u}_i, \mathbf{v}_j \rangle = \langle \mathbf{u}_i, \mathbf{w} \rangle$ for the partially orthogonalised $\mathbf{w}$, because the pieces already removed were orthogonal to $\mathbf{u}_i$.

In floating point they are not. The classical version computes every projection coefficient from the *original* $\mathbf{v}_j$, so an error made at step 1 is never seen again — it propagates untouched into every later step. The modified version recomputes against the running residual, so each step sees, and partially corrects, the error so far.

A **Hilbert matrix**, $h_{ij} = 1/(i+j+1)$, is famously ill-conditioned; run both on one:

```
n = 10
hilbert = np.array([[1.0 / (i + j + 1) for j in range(n)] for i in range(n)])
print('condition number     :', f'{np.linalg.cond(hilbert):.3e}')
print('classical error      :', orthogonality_error(gram_schmidt_classical(hilbert)))
print('modified error       :', orthogonality_error(gram_schmidt_modified(hilbert)))
print('numpy qr error       :', orthogonality_error(np.linalg.qr(hilbert)[0]))
```

The classical version loses orthogonality catastrophically; the modified one survives; LAPACK's QR (Householder reflections, a different algorithm entirely) is better still. **This is the lesson of Part I in a single experiment:** correct mathematics and correct numerics are two different achievements, and the gap between them widens exactly where your data is most structured.

2.5 Gate for this chapter

Check	Threshold
$\|(3,4)\|_2$	exactly 5.0
Your hand-worked Gram–Schmidt matches the code	to 1e-12
$Q^TQ = I$ for a well-conditioned input	max deviation < 1e-10
Classical vs modified on a 10×10 Hilbert matrix	classical error > modified by orders of magnitude
Projection is idempotent and symmetric	$P^2 = P$, $P = P^T$ to 1e-12

3 · Matrices as linear maps

3.1 The map is the point

A matrix is not a grid of numbers. It is a **linear map** written down in a particular basis *[MML p.117]*. A function T is linear when

$$T(ax + by) = a \cdot T(x) + b \cdot T(y)$$

and every linear map between finite-dimensional spaces *is* a matrix, once you fix bases *[MML p.121]*.

Why composition is matrix multiplication

The definition of matrix multiplication looks arbitrary until you derive it from composition, which takes three lines. Let B send basis vector $\mathbf{e}_j$ to the column $\Sigma_k\, b_{kj}\mathbf{e}_k$, and let A send $\mathbf{e}_k$ to $\Sigma_i\, a_{ik}\mathbf{e}_i$. Then

$$(A \circ B)(\mathbf{e}_j) = A(\Sigma_k\, b_{kj}\mathbf{e}_k) = \Sigma_k\, b_{kj} \cdot A(\mathbf{e}_k) = \Sigma_k\, b_{kj}\, \Sigma_i\, a_{ik}\mathbf{e}_i = \Sigma_i\, (\Sigma_k\, a_{ik} b_{kj})\, \mathbf{e}_i$$

The coefficient of $\mathbf{e}_i$ is $\Sigma_k\, a_{ik} b_{kj}$ — precisely the (i,j) entry of AB. ■

So the "row times column" rule is not a convention; it is what composition *forces*. Three consequences worth stating plainly:

- **AB means "do B, then A".** Non-commutativity is not a quirk: doing things in the other order is a different thing.
- **The columns are the images of the basis vectors.** $A\mathbf{e}_j$ is the j-th column. To know what a matrix *does*, look at where it sends the axes.
- **A neural network layer is a linear map plus a shift plus a nonlinearity.** Strip the nonlinearity and the deepest network collapses into one matrix — which is exactly why the nonlinearity is not optional.

3.2 The four quantities

Quantity	Definition	What it tells you
Image (column space)	$\{A\mathbf{x} : \mathbf{x} \in \mathbb{R}^n\}$	what outputs are reachable
Kernel (null space)	$\{\mathbf{x} : A\mathbf{x} = \mathbf{0}\}$	what inputs are destroyed
Rank	dim of the image	how much information survives
Nullity	dim of the kernel	how much is lost

tied together by **rank + nullity = n** *[MML p.127]*. The proof is a basis count: take a basis of the kernel, extend it to a basis of the whole space, and show the images of the added vectors form a basis of the image. The counting then gives the identity directly.

A nonzero kernel means the map is not invertible: distinct inputs collide, and no algorithm can undo that. When a linear model has a nonzero kernel in its design matrix, there is an entire subspace of coefficient vectors producing identical predictions — Section 1.2's three representations, in general form.

3.3 Transformations you can see

In the plane, every 2×2 matrix is some combination of five moves *[MML p.133–138]*:

Move	Matrix	Determinant
Stretch	[[a,0],[0,b]]	ab
Rotation by θ	[[cos θ, −sin θ],[sin θ, cos θ]]	1
Shear	[[1,k],[0,1]]	1
Reflection	[[1,0],[0,−1]]	−1
Projection	[[1,0],[0,0]]	0

Read the determinant column. Rotation and shear preserve area; reflection preserves area but flips orientation; projection destroys area entirely — and a determinant of zero is precisely the signature of information that cannot be recovered.

Worked example — the shear preserves area

Take the unit square with corners (0,0), (1,0), (1,1), (0,1) and apply the shear [[1,2],[0,1]]. The corners map to (0,0), (1,0), (3,1), (2,1): a slanted parallelogram. Its area is base × height = 1 × 1 = 1, unchanged, matching det = 1·1 − 2·0 = 1.

The shape is unrecognisable; the area is identical. That is what "determinant measures volume, not form" means in practice.

3.4 The determinant

The determinant is the **signed volume scaling factor** of the map *[MML p.139–140]*. Everything else about it follows from that one sentence:

- $\det(AB) = \det(A)\det(B)$ — scalings compose by multiplying;
- $\det(A) = 0 \iff A$ is singular — the map flattens space, so it cannot be undone;
- $|\det(A)|$ is the factor by which any region's volume changes;
- a negative sign means orientation was flipped;
- $\det(A^{-1}) = 1/\det(A)$, since undoing a scaling scales by the reciprocal.

In practice you never compute it by cofactor expansion — that is O(n!) and unusable past n ≈ 12 *[MML p.180]*. You factor the matrix first and multiply the diagonal, which is Chapter 4.

3.5 Gate for this chapter

Check	Threshold
Your 2×2 gallery reproduces the determinant column above	exact
rank + nullity = n on random rank-deficient matrices	exact, 100 trials
Rotation preserves $\|\mathbf{x}\|_2$	within 1e-12
The sheared unit square has area 1	exact

4 · Solving linear systems

4.1 Elimination

Solving A**x** = **b** by hand means **Gaussian elimination**: use row operations to reach an upper-triangular system, then back-substitute *[MML p.160]*.

Worked example — elimination with numbers

Solve

$$2x + y - z = 8 \;\cdot\; -3x - y + 2z = -11 \;\cdot\; -2x + y + 2z = -3$$

Eliminate x using row 1 (pivot 2). Row 2 ← Row 2 + 1.5·Row 1; Row 3 ← Row 3 + Row 1:

$$2x + y - z = 8 \;\cdot\; 0.5y + 0.5z = 1 \;\cdot\; 2y + z = 5$$

Eliminate y using the new row 2 (pivot 0.5). Row 3 ← Row 3 − 4·Row 2:

$$2x + y - z = 8 \;\cdot\; 0.5y + 0.5z = 1 \;\cdot\; -z = 1$$

Back-substitute: z = −1, then 0.5y − 0.5 = 1 so y = 3, then 2x + 3 + 1 = 8 so x = 2. **(x, y, z) = (2, 3, −1)**, which you can verify in the original equations in ten seconds.

Count the work: roughly $n^3/3$ multiply-adds, i.e. **O(n³)** *[MML p.165]*. That single exponent explains a surprising amount of practical machine learning — doubling the feature count of an exact linear solve costs eight times the work.

Why pivoting is not optional

Consider, in three-digit arithmetic:

$$0.001x + y = 1 \;\cdot\; x + y = 2$$

Without pivoting, the multiplier is 1/0.001 = 1000, and row 2 becomes (1 − 1000)y = 2 − 1000, i.e. −999y = −998, giving y = 0.999 and then x = (1 − 0.999)/0.001 = 1. Rounded to three digits, y = 1.00 and x = (1 − 1.00)/0.001 = **0** — catastrophically wrong.

Swap the rows first (partial pivoting puts the largest available pivot on the diagonal) and the multiplier is 0.001, the arithmetic is benign, and you get x ≈ 1.00, y ≈ 1.00. Same mathematics, different numerics, completely different answer.

4.2 LU

Elimination can be *recorded* rather than merely performed. The record is the **LU decomposition** *[MML p.168, code p.171]*:

$$PA = LU$$

P a permutation (the row swaps), L unit lower-triangular (the multipliers), U upper-triangular (the result). Having it, each new right-hand side costs only $O(n^2)$: solve $L\mathbf{y} = P\mathbf{b}$ forwards, then $U\mathbf{x} = \mathbf{y}$ backwards. This is the standard pattern — **factor once, solve many times** — and it is why `scipy.linalg.lu_factor` exists separately from `lu_solve`.

For the worked example above, the multipliers were -1.5 and -1 (first column) and 4 (second), so

$$L = [[1,0,0],[-1.5,1,0],[-1,4,1]], \quad U = [[2,1,-1],[0,0.5,0.5],[0,0,-1]]$$

and LU reproduces A exactly. Multiply them out once by hand; it is the clearest possible demonstration that L and U are just elimination's bookkeeping.

```python
"""LU decomposition with partial pivoting, and solving with it."""

import numpy as np


def lu_decompose(a: np.ndarray) -> tuple:
    """Factors a square matrix as PA = LU with partial pivoting.

    Args:
        a: Square matrix of shape (n, n).

    Returns:
        A tuple (P, L, U) of ndarrays.

    Raises:
        ValueError: If `a` is not square or is singular.
    """
    if a.ndim != 2 or a.shape[0] != a.shape[1]:
        raise ValueError(f'expected a square matrix, got {a.shape}')
    n = a.shape[0]
    u = a.astype(np.float64).copy()
    l = np.eye(n)
    p = np.eye(n)
    for k in range(n - 1):
        pivot = int(np.argmax(np.abs(u[k:, k]))) + k   # partial pivoting
        if abs(u[pivot, k]) < 1e-14:
            raise ValueError('matrix is singular to working precision')
        if pivot != k:
            u[[k, pivot], k:] = u[[pivot, k], k:]
            l[[k, pivot], :k] = l[[pivot, k], :k]
            p[[k, pivot], :] = p[[pivot, k], :]
        for row in range(k + 1, n):
            factor = u[row, k] / u[k, k]
            l[row, k] = factor
            u[row, k:] -= factor * u[k, k:]
    return p, l, u


def lu_solve(p: np.ndarray, l: np.ndarray, u: np.ndarray,
             b: np.ndarray) -> np.ndarray:
    """Solves Ax = b given a PA = LU factorisation.

    Args:
        p: Permutation matrix.
        l: Unit lower-triangular factor.
        u: Upper-triangular factor.
        b: Right-hand side of shape (n,).

    Returns:
        The solution vector x.
    """
    pb = p @ b
    n = len(b)
    y = np.zeros(n)
    for i in range(n):                       # forward substitution
        y[i] = pb[i] - l[i, :i] @ y[:i]
    x = np.zeros(n)
    for i in range(n - 1, -1, -1):           # back substitution
        x[i] = (y[i] - u[i, i + 1:] @ x[i + 1:]) / u[i, i]
    return x
```

The determinant now costs nothing extra: det(A) = (−1)^swaps · Π u_{ii} *[MML p.182]*. In the example, det = $(-1)^0 \cdot 2 \cdot 0.5 \cdot (-1) = \mathbf{-1}$.

4.3 Never invert

The single most common numerical error in applied work is computing A^{-1} and multiplying *[MML p.177]*. Inversion costs about three times a solve, and it loses accuracy: it computes n separate solves and then makes you push the result through a matrix multiply, compounding each error.

```
rng = np.random.default_rng(0)
n = 200
base = rng.normal(size=(n, n))
# Bend the spectrum to make the system badly conditioned on purpose.
u_mat, _, vt = np.linalg.svd(base)
spectrum = np.logspace(0, -10, n)
a = (u_mat * spectrum) @ vt
x_true = rng.normal(size=n)
b = a @ x_true

x_inv = np.linalg.inv(a) @ b
x_solve = np.linalg.solve(a, b)
print(f'condition number : {np.linalg.cond(a):.3e}')
print(f'error via inv    : {np.linalg.norm(x_inv - x_true):.3e}')
print(f'error via solve  : {np.linalg.norm(x_solve - x_true):.3e}')
```

The condition number, derived

Why does conditioning amplify error? Perturb the right-hand side by $\delta\mathbf{b}$. The solution moves by $\delta\mathbf{x} = A^{-1}\delta\mathbf{b}$, so

$$\|\delta x\| \le \|A^{-1}\|\cdot\|\delta b\| \quad \text{and} \quad \|b\| = \|Ax\| \le \|A\|\cdot\|x\|$$

Divide the first by $\|\mathbf{x}\|$ and use the second:

$$\|\delta x\|/\|x\| \le \|A\|\cdot\|A^{-1}\| \cdot \|\delta b\|/\|b\| = \kappa(A)\cdot\|\delta b\|/\|b\|$$

So **$\kappa(A) = \|A\|\|A^{-1}\| = \sigma_{max}/\sigma_{min}$ is exactly the worst-case amplification factor** from relative input error to relative output error. ■

A useful rule of thumb follows: at $\kappa \approx 10^k$ you lose about k significant digits. Double precision carries ~16, so $\kappa \approx 10^{16}$ leaves you none. When a linear model's coefficients swing wildly for an invisible change in the data, this is the mechanism — and the fix is regularisation (Chapter 12), which raises σ_{min} off the floor.

4.4 Least squares, derived

An over-determined system — n observations, $d < n$ unknowns — has no exact solution, so you ask for the **closest** one: minimise $\|A\mathbf{x} - \mathbf{b}\|_2$. Two derivations, and it is worth seeing both.

Geometrically. The achievable vectors $A\mathbf{x}$ form the column space. The closest point in a subspace is the orthogonal projection (Section 2.3), so the residual must be orthogonal to *every* column of A:

$$A^T(b - Ax) = 0 \implies A^TAx = A^Tb$$

By calculus. Expand the objective and differentiate:

$$f(x) = \|Ax - b\|^2 = x^TA^TAx - 2b^TAx + b^Tb$$

$$\nabla f = 2A^TAx - 2A^Tb = 0 \implies A^TAx = A^Tb$$

The same **normal equations**, reached from geometry and from calculus. This is linear regression, and you will meet it again as a maximum-likelihood estimate in Chapter 8, as ridge regression in Chapter 12, and as the first thing a neural network learns to imitate in Chapter 18.

One warning that costs real accuracy: $\kappa(A^TA) = \kappa(A)^2$. Forming the normal equations *squares* your conditioning problem. For well-behaved data it is fine and fast; for anything delicate, solve via QR or SVD instead, which is what `np.linalg.lstsq` does for you.

4.5 Gate for this chapter

Check	Threshold
Your hand-solved 3×3 system	(2, 3, −1) exactly
‖PA − LU‖	< 1e-10
Your `lu_solve` against `np.linalg.solve`	agree to 1e-10
det via LU against `np.linalg.det`	agree to 1e-10 relative
`inv`-and-multiply vs `solve` on $\kappa \approx 10^{10}$	solve is measurably more accurate
Elimination timing at n = 64, 128, 256	fitted exponent ≈ 3

5 · Eigenvalues, the spectral theorem, and the SVD

5.1 Eigenvectors

$\mathbf{v} \neq \mathbf{0}$ is an **eigenvector** of A with **eigenvalue** λ when

$$A\mathbf{v} = \lambda\mathbf{v}$$

[MML p.189]: a direction the map does not rotate, only scales. Find them in principle through the characteristic polynomial $\det(A - \lambda I) = 0$ *[MML p.192]* — and never in practice past tiny matrices, because polynomial root-finding is catastrophically unstable. Real algorithms iterate instead *[MML p.217, p.230]*.

Worked example — a 2×2 by hand

Take $A = [[4, 1], [2, 3]]$. The characteristic polynomial:

$$\det(A - \lambda I) = (4-\lambda)(3-\lambda) - 2 = \lambda^2 - 7\lambda + 10 = (\lambda - 5)(\lambda - 2)$$

So $\lambda_1 = 5$, $\lambda_2 = 2$. For $\lambda = 5$, solve $(A - 5I)\mathbf{v} = \mathbf{0}$:

$$[[-1, 1], [2, -2]]\mathbf{v} = 0 \implies v_1 = v_2 \implies \mathbf{v}_1 \propto (1, 1)$$

For $\lambda = 2$: $[[2,1],[2,1]]\mathbf{v} = \mathbf{0}$ gives $2v_1 = -v_2$, so $\mathbf{v}_2 \propto (1, -2)$. Check: $A(1,1) = (5,5) = 5(1,1)$ ✓.

Note $\det(A) = 10 = 5 \times 2$ and $\text{trace}(A) = 7 = 5 + 2$. Both hold in general — **the determinant is the product of the eigenvalues and the trace is their sum** — which gives you a free sanity check on any eigenvalue computation.

Power iteration, and why it converges

Repeatedly multiply and renormalise. Why does it find the dominant eigenvector? Write the start vector in the eigenbasis, $\mathbf{x}_0 = \Sigma_i\, c_i\mathbf{v}_i$. Then

$$A^k\mathbf{x}_0 = \Sigma_i\, c_i\lambda_i{}^k\mathbf{v}_i = \lambda_1{}^k[\, c_1\mathbf{v}_1 + \Sigma_{i>1}\, c_i(\lambda_i/\lambda_1)^k\mathbf{v}_i\,]$$

Every ratio $|\lambda_i/\lambda_1| < 1$, so every term except the first decays geometrically. **The convergence rate is $|\lambda_2/\lambda_1|$** — fast when the dominant eigenvalue is well separated, painfully slow when $\lambda_2 \approx \lambda_1$. ■

```python
"""Power iteration for the dominant eigenpair of a symmetric matrix."""

import numpy as np


def power_iteration(a: np.ndarray, iters: int = 5000,
                    tol: float = 1e-13) -> tuple:
    """Finds the dominant eigenpair.

    Args:
        a: Symmetric matrix of shape (n, n).
        iters: Maximum number of iterations.
        tol: Stop when the iterate moves less than this.

    Returns:
        A tuple (eigenvalue, eigenvector) with the vector normalised.
    """
    rng = np.random.default_rng(0)
    v = rng.normal(size=a.shape[0])
    v /= np.linalg.norm(v)
    for _ in range(iters):
        w = a @ v
        norm = np.linalg.norm(w)
        if norm < 1e-300:
            return 0.0, v
        w /= norm
        if np.linalg.norm(w - v) < tol or np.linalg.norm(w + v) < tol:
            v = w
            break
        v = w
    return float(v @ a @ v), v          # Rayleigh quotient
```

The returned value $\mathbf{v}^{\mathrm{T}}A\mathbf{v}$ (for unit $\mathbf{v}$) is the **Rayleigh quotient**, and the largest eigenvalue is exactly its maximum over the unit sphere *[MML p.207, Thm 7.2.3]*.

Why the maximum of the Rayleigh quotient is λ_max

This is the fact that makes PCA an optimisation problem with a closed-form answer, so it deserves a proof. Let A be symmetric with orthonormal eigenvectors $\mathbf{u}_i$ and eigenvalues $\lambda_1 \geq \cdots \geq \lambda_n$. Write any unit $\mathbf{x}$ as $\Sigma_i\, x_i \mathbf{u}_i$, where $\Sigma_i\, x_i^2 = 1$. Then

$$\mathbf{x}^{\mathrm{T}}A\mathbf{x} = \Sigma_i\, x_i^2 \lambda_i \leq \lambda_1\, \Sigma_i\, x_i^2 = \lambda_1$$

with equality when $\mathbf{x} = \mathbf{u}_1$. So the maximum is λ_1, attained at the top eigenvector. ■ Restricting $\mathbf{x}$ to be orthogonal to $\mathbf{u}_1$ repeats the argument one level down and gives λ_2 — which is why principal components come out in order.

5.2 The spectral theorem

Every real **symmetric** matrix can be diagonalised by an orthogonal matrix *[MML p.203]*:

$$A = Q\Lambda Q^{\mathrm{T}}, \quad Q^{\mathrm{T}}Q = I, \quad \Lambda = \mathrm{diag}(\lambda_1, \ldots, \lambda_n)$$

Two parts of this are worth proving, because both are used constantly.

Eigenvalues are real. Suppose $A\mathbf{v} = \lambda\mathbf{v}$ with possibly complex λ and $\mathbf{v}$. Take the conjugate transpose: $\mathbf{v}^*A = \bar{\lambda}\mathbf{v}^*$. Then

$$\mathbf{v}^*A\mathbf{v} = \lambda\mathbf{v}^*\mathbf{v} \quad \text{and} \quad \mathbf{v}^*A\mathbf{v} = \bar{\lambda}\mathbf{v}^*\mathbf{v}$$

Since $\mathbf{v}^*\mathbf{v} > 0$, $\lambda = \bar{\lambda}$, so λ is real. ■

Eigenvectors of distinct eigenvalues are orthogonal. With $A\mathbf{u} = \lambda\mathbf{u}$ and $A\mathbf{w} = \mu\mathbf{w}$, $\lambda \neq \mu$:

$$\lambda\langle \mathbf{u},\mathbf{w}\rangle = \langle A\mathbf{u},\mathbf{w}\rangle = \langle \mathbf{u},A\mathbf{w}\rangle = \mu\langle \mathbf{u},\mathbf{w}\rangle$$

so $(\lambda - \mu)\langle \mathbf{u},\mathbf{w}\rangle = 0$, and since $\lambda \neq \mu$ the inner product vanishes. ■

This matters more than it looks, because the matrices that actually arise in machine learning are symmetric almost without exception: covariance matrices, Gram matrices, kernel matrices, Hessians. **A symmetric matrix is just a scaling along orthogonal axes, viewed from a rotated frame.**

5.3 The singular value decomposition

Non-square, non-symmetric matrices — real data — get the SVD. The derivation in *[MML p.207]* is pure wishful thinking, and it is worth following because it shows where the pieces come from. Suppose

$$A = U\Sigma V^T$$

with $U \in \mathbb{R}^{n\times n}$, $V \in \mathbb{R}^{m\times m}$ orthogonal and $\Sigma \in \mathbb{R}^{n\times m}$ diagonal. Then

$$AA^T = U\Sigma V^T V\Sigma^T U^T = U\Sigma^2 U^T \quad \text{and} \quad A^TA = V\Sigma^2 V^T$$

Both are symmetric, so the spectral theorem hands them to us: $\mathbf{U}$ is the eigenvectors of AA^T, $\mathbf{V}$ the eigenvectors of A^TA, and the **singular values** σ_i are the square roots of their shared eigenvalues — non-negative, because A^TA is positive semi-definite ($\mathbf{x}^TA^TA\mathbf{x} = \|A\mathbf{x}\|^2 \geq 0$). The decomposition exists for *every* matrix *[MML p.208, Thm 7.3.1]*.

Worked example — the SVD of a 2×2

Take $A = [[3, 0], [4, 5]]$. Then

$$A^TA = [[25, 20], [20, 25]]$$

whose eigenvalues solve $(25-\lambda)^2 = 400$, giving $\lambda = 45$ and $\lambda = 5$. So the singular values are $\sigma_1 = \sqrt{45} \approx 6.708$ and $\sigma_2 = \sqrt{5} \approx 2.236$. The eigenvectors of A^TA are $(1,1)/\sqrt{2}$ and $(1,-1)/\sqrt{2}$ — these are the columns of V.

Check against the determinant: |det A| = 15, and $\sigma_1\sigma_2 = \sqrt{45}\cdot\sqrt{5} = \sqrt{225} = 15$ ✓. **The product of the singular values is the absolute determinant**, which is the volume statement of Chapter 3 restated in the SVD's language.

Geometrically every linear map is therefore: **rotate (V^T), scale along axes (Σ), rotate again (U)**. Nothing else is possible. A is sending the unit circle to an ellipse with semi-axes 6.708 and 2.236.

5.4 Low rank, and why it is the whole game

Truncate to the k largest singular values and you get the best rank-k approximation that exists, in both the Frobenius and spectral norms — the **Eckart–Young theorem**. The error is exactly the norm of what you discarded:

$$\|A - A_k\|_F = \sqrt{(\sigma^2{}_{k+1} + \cdots + \sigma^2{}_r)}$$

Why the error takes that form

The Frobenius norm is unitarily invariant: ||UMV^T||_F = ||M||_F, because orthogonal matrices preserve lengths. So

$$\|A - A_k\|_F = \|U(\Sigma - \Sigma_k)V^T\|_F = \|\Sigma - \Sigma_k\|_F$$

and $\Sigma - \Sigma_k$ is the diagonal matrix holding exactly the discarded singular values, whose Frobenius norm is the root-sum-of-squares of them. ■ (That the truncation is *optimal* among all rank-k matrices is the harder half of Eckart–Young; the error identity above is the half you will verify numerically.)

```python
"""Low-rank approximation and the Eckart-Young error identity."""

import numpy as np


def svd_lowrank(a: np.ndarray, k: int) -> np.ndarray:
    """Returns the best rank-k approximation of a matrix.

    Args:
        a: Matrix to approximate.
        k: Target rank.

    Returns:
        The rank-k approximation, with the same shape as `a`.

    Raises:
        ValueError: If k is not in [1, min(a.shape)].
    """
    if not 1 <= k <= min(a.shape):
        raise ValueError(f'k must lie in [1, {min(a.shape)}], got {k}')
    u, s, vt = np.linalg.svd(a, full_matrices=False)
    return (u[:, :k] * s[:k]) @ vt[:k, :]


rng = np.random.default_rng(0)
a = rng.normal(size=(60, 30))
s = np.linalg.svd(a, compute_uv=False)
for k in (1, 5, 10, 20):
    predicted = float(np.sqrt(np.sum(s[k:] ** 2)))
    actual = float(np.linalg.norm(a - svd_lowrank(a, k)))
    print(f'k={k:>3}  predicted {predicted:9.5f}  actual {actual:9.5f}  '
          f'match {abs(predicted - actual) < 1e-10}')
```

That identity is not a curiosity. It is the reason compression works, the reason PCA keeps what it keeps, the reason embeddings have the dimensions they do, and the reason LoRA can fine-tune a billion-parameter model by training two thin matrices.

5.5 PCA, derived three ways

PCA is usually presented as a recipe. It is better understood as the *same answer* arrived at from three directions — and seeing that is what makes it stop being a black box.

(a) Maximum variance. Find the unit direction $\mathbf{w}$ maximising the variance of the projected data. With centred X, the projected variance is

$$\mathrm{Var}(\mathrm{X}\mathrm{w}) = (1/(\mathrm{n}-1))\|\mathrm{X}\mathrm{w}\|^2 = \mathrm{w}^{\mathrm{T}}\,\mathrm{C}\,\mathrm{w}, \quad \mathrm{C} = \mathrm{X}^{\mathrm{T}}\mathrm{X}/(\mathrm{n}-1)$$

Maximise $\mathbf{w}^{\mathrm{T}}\mathrm{C}\mathbf{w}$ subject to $\|\mathbf{w}\| = 1$. Lagrange:

$$\nabla[\mathrm{w}^{\mathrm{T}}\mathrm{C}\mathrm{w} - \lambda(\mathrm{w}^{\mathrm{T}}\mathrm{w} - 1)] = 2\mathrm{C}\mathrm{w} - 2\lambda\mathrm{w} = 0 \implies \mathrm{C}\mathrm{w} = \lambda\mathrm{w}$$

The stationary points are the **eigenvectors of the covariance matrix**, and by Section 5.1's Rayleigh argument the maximum is the top one. ■

(b) Minimum reconstruction error. Find the k-dimensional subspace minimising $\Sigma_i \|\mathbf{x}_i - \text{proj}(\mathbf{x}_i)\|^2$. Since $\|\mathbf{x}\|^2 = \|\text{proj}\|^2 + \|\text{residual}\|^2$ (Pythagoras, because the projection is orthogonal), minimising the residual is *identical* to maximising the projected variance. Same problem, same answer.

(c) The SVD. Centre, take $X = U\Sigma V^T$. Then $C = X^TX/(n-1) = V\Sigma^2V^T/(n-1)$, so V holds the eigenvectors of C and the explained variance of component i is $\sigma_i^2/(n-1)$. Computing the SVD of X directly is numerically better than forming C first, for exactly the $\kappa(X^TX) = \kappa(X)^2$ reason from Chapter 4.

```python
def pca(x: np.ndarray, n_components: int) -> tuple:
    """Computes PCA directly from the SVD of the centred data.

    Args:
        x: Data matrix of shape (n_samples, n_features).
        n_components: Number of components to keep.

    Returns:
        A tuple (scores, components, explained_variance_ratio).
    """
    centred = x - x.mean(axis=0)
    u, s, vt = np.linalg.svd(centred, full_matrices=False)
    variance = s ** 2 / (x.shape[0] - 1)
    scores = (u[:, :n_components] * s[:n_components])
    return scores, vt[:n_components], variance[:n_components] / variance.sum()
```

Two consequences people learn the hard way. **Centring is not optional** — derivation (a) assumed it when it wrote the variance as $\mathbf{w}^T\mathbf{C}\mathbf{w}$; skip it and the first component points at the mean. And **scale matters**: PCA maximises variance, and variance has units, so a feature in millimetres will dominate the same feature in metres. Standardise unless the units are genuinely comparable.

5.6 Gate for this chapter

Check	Threshold
Your hand-computed eigenvalues of [[4,1],[2,3]]	5 and 2 exactly
trace = $\Sigma\lambda$ and det = $\Pi\lambda$	to 1e-12
Your hand-computed singular values of [[3,0],[4,5]]	$\sqrt{45}$ and $\sqrt{5}$
`power_iteration` against `np.linalg.eigvalsh().max()`	relative 1e-6
$\|A - Q\Lambda Q^T\|$ for symmetric A	< 1e-10
Eckart–Young identity at k = 1, 5, 10, 20	< 1e-10 absolute
Your PCA against the covariance eigendecomposition	ratios agree to 1e-8

6 · Derivatives and optimisation in one variable

6.1 Three readings of one number

The derivative $f'(x)$ is the limit of the difference quotient, and it is simultaneously the instantaneous rate of change, the slope of the tangent, and the coefficient of the best linear approximation at x *[MML p.343]*. For optimisation there is a fourth reading, and it is the one that matters: **f′(x) is a one-dimensional vector pointing towards increase** *[MML p.382]*. Positive means "uphill to the right", negative means "uphill to the left", and the magnitude is the steepness.

Hold on to that, because gradient descent is nothing more than taking that sentence seriously in n dimensions.

6.2 Differentiating numerically

You will often need a derivative you have not derived — to check an analytic gradient, or to probe a black box. The forward difference is the obvious choice and the wrong one. Taylor tells you why:

$$f(x+h) = f(x) + hf'(x) + (h^2/2)f''(x) + O(h^3)$$

Rearranging gives the forward difference with an error of $(h/2)f''(x)$ = **O(h)**. Now expand in both directions and subtract:

$$f(x+h) - f(x-h) = 2hf'(x) + (h^3/3)f'''(x) + O(h^5)$$

$$(f(x+h) - f(x-h))/2h = f'(x) + (h^2/6)f'''(x) + O(h^4)$$

The second-order term **cancels**, leaving error **O(h²)**. One extra function evaluation buys a whole order of accuracy — that is why every gradient check in deep learning uses central differences.

The error floor, worked out

Smaller h is not monotonically better, and this trips up almost everyone the first time. Two errors move in opposite directions:

- **truncation** falls as h shrinks: $\approx (h^2/6)|f'''|$;
- **round-off** *grows* as h shrinks: the numerator is a difference of two nearly equal floats, each carrying absolute error $\approx \varepsilon|f|$, and you then divide by 2h, giving $\approx \varepsilon|f|/h$.

Total error $\approx (h^2/6)|f'''| + \varepsilon|f|/h$. Differentiate with respect to h and set to zero:

$$(h/3)|f'''| - \varepsilon|f|/h^2 = 0 \implies h^3 = 3\varepsilon|f|/|f'''| \implies h \propto \varepsilon^{1/3}$$

With $\varepsilon \approx 2.2\times10^{-16}$, that is $\mathbf{h \approx 6\times10^{-6}}$ — not the smallest representable number, and the resulting error floor is around 10^{-11}, not 10^{-16}.

```python
"""The U-shaped error curve of numerical differentiation."""

import numpy as np


def central_difference(f, x: float, h: float) -> float:
    """Approximates f'(x) by a central difference.

    Args:
        f: Scalar function of one variable.
        x: Point of evaluation.
        h: Step size.

    Returns:
        The approximate derivative.
    """
    return (f(x + h) - f(x - h)) / (2.0 * h)


f = np.sin
exact = np.cos(1.0)
print(f'{"h":>10}  {"estimate":>18}  {"abs error":>12}')
for power in range(1, 16):
    h = 10.0 ** (-power)
    got = central_difference(f, 1.0, h)
    print(f'{h:>10.0e}  {got:>18.12f}  {abs(got - exact):>12.3e}')
```

Run it and read the column: the error falls, bottoms out near 1e-11 around h = 1e-5 or 1e-6, then *climbs again* — exactly where the algebra above said it would.

6.3 Gradient descent

Minimising by solving $f'(x) = 0$ fails in practice for two reasons: the stationary points include maxima and inflections, and the equation is unsolvable except in trivial cases — linear regression with squared error being the notable exception *[MML p.381]*. So we iterate instead.

To *maximise*, step along the derivative. To **minimise**, step against it *[MML p.383–384]*:

$$x_{n+1} := x_n - h \cdot f'(x_n)$$

where $h > 0$ is the **learning rate**. That is the entire algorithm, and every optimiser in Part III is a variation on it.

Why a step downhill reduces the loss: the descent lemma

The mountain analogy says "downhill is good" but not "how far". Taylor with a remainder does. Suppose the gradient is L-Lipschitz, meaning $|f'(a) - f'(b)| \le L|a - b|$ — curvature is bounded by L. Then

$$f(y) \le f(x) + f'(x)(y - x) + (L/2)(y - x)^2$$

Substitute the gradient step $y = x - hf'(x)$:

$$f(x - hf'(x)) \le f(x) - h \cdot f'(x)^2 + (L/2)h^2 f'(x)^2 = f(x) - h(1 - Lh/2) \cdot f'(x)^2$$

The loss *decreases* whenever $h(1 - Lh/2) > 0$, that is whenever

$$0 < h < 2/L$$

and the guaranteed decrease is largest at $h = 1/L$. ■

This is the entire theory of learning rates in two lines. It also tells you what L means operationally: the largest curvature your loss has anywhere you travel.

The same result on a quadratic, exactly

Let $f(x) = (a/2)x^2$ with $a > 0$, so $f'(x) = ax$ and $L = a$. Then

$$x_{n+1} = x_n - h \cdot a \cdot x_n = (1 - ha) \cdot x_n \implies x_n = (1 - ha)^n \cdot x_0$$

Convergence iff $|1 - ha| < 1$, i.e. $0 < h < 2/a$ — matching the lemma, now with no inequality slack. Four regimes, which you should be able to recognise from a loss curve alone:

Learning rate	Factor $(1 - ha)$	Behaviour
$h < 1/a$	in $(0, 1)$	smooth monotone convergence
$h = 1/a$	0	converges in **one step**
$1/a < h < 2/a$	in $(-1, 0)$	converges, oscillating in sign
$h = 2/a$	-1	perpetual oscillation, no progress
$h > 2/a$	< -1	**divergence**, growing geometrically

Worked iterates

Take $a = 2$ (so $L = 2$, the limit is $h = 1$) and $x_0 = 1$. Three learning rates, five steps each:

n	h = 0.25	h = 0.5 (=1/a)	h = 1.1
0	1.000	1.000	1.000
1	0.500	0.000	−1.200
2	0.250	0.000	1.440
3	0.125	0.000	−1.728
4	0.0625	0.000	2.074
5	0.03125	0.000	−2.488

Halving per step; exact in one step; and alternating while growing by 1.2× per step. The third column is divergence — and notice it *oscillates*, which is the practical signature: a loss that zigzags upward means the step is past 2/L, not that the data is bad.

In many dimensions

The Hessian replaces a. Diagonalise it (it is symmetric — Chapter 5): along eigenvector i the iteration contracts by $(1 - h\lambda_i)$. So:

- **stability** is set by the *largest* eigenvalue: h < 2/λ_max;
- **speed** is set by the *smallest*: the slowest direction contracts by (1 − hλ_min) per step.

With h just under 2/λ_max, the slow direction's factor is 1 − 2λ_min/λ_max = 1 − 2/κ. **The number of iterations scales with the condition number κ.** That is the zig-zag you have seen in contour plots, and it is why feature scaling is not cosmetic: it is conditioning the Hessian.

```python
"""Gradient descent, and the stability limit h < 2/L."""

import numpy as np


def gradient_descent(grad, x0: np.ndarray, lr: float, iters: int,
                     momentum: float = 0.0) -> np.ndarray:
    """Minimises a function given its gradient.

    Divergence above the stability limit is a real outcome and is not
    clamped away: a routine that cannot diverge cannot teach you where
    the limit is.

    Args:
        grad: Function returning the gradient at a point.
        x0: Starting point.
        lr: Learning rate.
        iters: Number of steps.
        momentum: Heavy-ball coefficient; 0.0 is plain descent.

    Returns:
        The final iterate.
    """
    x = np.asarray(x0, dtype=np.float64).copy()
    velocity = np.zeros_like(x)
    for _ in range(iters):
        velocity = momentum * velocity - lr * np.asarray(grad(x), dtype=float)
        x = x + velocity
        if not np.all(np.isfinite(x)):
            break
    return x


curvature = np.array([4.0, 1.0])            # L = 2 * max(curvature) = 8
limit = 2.0 / (2.0 * curvature.max())       # stability limit on h
grad = lambda x: 2.0 * curvature * x        # f(x) = Σ aᵢxᵢ²

for scale in (0.1, 0.5, 0.99, 1.01, 1.5):
    lr = scale * limit
    final = gradient_descent(grad, np.array([5.0, -3.0]), lr, 500)
    size = np.linalg.norm(final)
    verdict = 'converged' if np.isfinite(size) and size < 1e-6 else (
        'DIVERGED' if not np.isfinite(size) or size > 1e3 else 'crawling')
    print(f'h = {lr:7.4f} ({scale:4.2f} x limit)   ‖x‖ = {size:9.3e}   {verdict}')
```

Gradient descent is a discretised ODE

There is a deeper reading *[MML p.389]*. Consider the **gradient flow**

$$x'(t) = -f'(x(t))$$

a trajectory that always moves downhill. Its forward-Euler discretisation with step h is x_{n+1} = x_n − h f′(x_n) — gradient descent, exactly. So the iteration is a numerical approximation of a continuous descent, and everything known about unstable step sizes in numerical integration applies to learning rates. The divergence above 2/L is the same instability that makes Euler blow up on a stiff ODE, and "stiffness" is just another word for a large condition number.

Momentum

Add memory: v_{n+1} = βv_n − h∇f, x_{n+1} = x_n + v_{n+1}. On a quadratic, a component with curvature λ now follows a *second-order* recurrence whose behaviour is governed by the roots of

$$z^2 - (1 + \beta - h\lambda)z + \beta = 0$$

Oscillating components (which flip sign every step) cancel across steps while consistent ones accumulate — the effective step along a persistent direction is multiplied by about 1/(1 − β). At β = 0.9 that is a tenfold amplification of the slow direction, which is why momentum so often turns a crawl into a sprint on ill-conditioned problems.

6.4 Gate for this chapter

Check	Threshold
Central-difference error curve	minimum near h ≈ 1e-5, then rises
Your worked iterate table at a = 2	matches to 1e-12
Convergence for h < 2/L	‖x‖ < 1e-6
Divergence for h > 2/L	required observation, recorded
Momentum vs plain on curvature ratio 4:1	fewer iterations to tolerance

7 · Many variables: gradients, Jacobians, backpropagation

7.1 Gradient

For $f : \mathbb{R}^n \to \mathbb{R}$, the **partial derivative** $\partial f/\partial x_i$ differentiates in one coordinate holding the rest fixed, and the **gradient** collects them *[MML p.448]*:

$$\nabla f(\mathbf{x}) = (\partial f/\partial x_1, \ldots, \partial f/\partial x_n)$$

Why the gradient is the steepest direction

The directional derivative along a unit vector $\mathbf{u}$ is

$$D_{\mathbf{u}} f = \lim_{t \to 0} (f(\mathbf{x} + t\mathbf{u}) - f(\mathbf{x}))/t = \langle \nabla f, \mathbf{u} \rangle$$

By Cauchy–Schwarz, $\langle \nabla f, \mathbf{u} \rangle \le \|\nabla f\| \cdot \|\mathbf{u}\| = \|\nabla f\|$, with equality precisely when $\mathbf{u}$ points along ∇f. So the gradient is the direction of **steepest ascent**, its negative the steepest descent, and its norm is the steepness. ■

That single inequality is what licenses Chapter 6's algorithm in n dimensions *[MML p.481]*.

7.2 Jacobian and the chain rule

For a vector-valued $f : \mathbb{R}^n \to \mathbb{R}^m$, the derivative is the **Jacobian**, the m×n matrix of all partials *[MML p.460]*. The chain rule then becomes matrix multiplication:

$$J_{f \circ g}(\mathbf{x}) = J_f(g(\mathbf{x})) \cdot J_g(\mathbf{x})$$

Read that line twice, because **it is backpropagation**. A neural network is a composition $f_L \circ \cdots \circ f_1$; the gradient of the loss is a product of Jacobians; and the only question is the order in which you multiply them.

The cost argument for reverse mode

Consider a chain $\mathbb{R}^n \to \mathbb{R}^{h} \to \mathbb{R}^{h} \to \mathbb{R}$, with Jacobians J_3 (1×h), J_2 (h×h), J_1 (h×n). The product $J_3 J_2 J_1$ can be bracketed two ways:

Order	Operations	Intermediate
$(J_3J_2)J_1$ — **reverse**	$h^2 + hn$	a 1×h row vector
$J_3(J_2J_1)$ — forward	$h^2n + hn$	an h×n matrix

Reverse mode is cheaper by a factor of about n — the number of inputs. Since a loss has **one output** and millions of inputs, reverse mode wins overwhelmingly, and reverse mode *is* backpropagation. The cost of a gradient is then a small constant times the cost of a forward pass, independently of the parameter count. **Every deep model in existence rests on that asymmetry.**

7.3 Backpropagation, derived in full

Do this once by hand and autograd stops being magic. Two layers, scalar output, squared error:

$$z_1 = W_1x + b_1 \;\cdot\; a_1 = \sigma(z_1) \;\cdot\; z_2 = W_2a_1 + b_2 \;\cdot\; L = \tfrac{1}{2}(z_2 - y)^2$$

Work backwards, defining $\delta = \partial L/\partial$(pre-activation) at each layer.

Output layer. L is an explicit function of z_2:

$$\delta_2 = \partial L/\partial z_2 = (z_2 - y)$$

Since $z_2 = W_2a_1 + b_2$, and $\partial z_2/\partial W_2[i,j] = a_1[j]$:

$$\partial L/\partial W_2 = \delta_2 a_1^{T} \;\cdot\; \partial L/\partial b_2 = \delta_2$$

Hidden layer. The loss reaches z_1 only through a_1 and then z_2, so the chain rule gives

$$\partial L/\partial a_1 = W_2^{T}\delta_2 \quad \text{then} \quad \delta_1 = \partial L/\partial z_1 = (W_2^{T}\delta_2) \odot \sigma'(z_1)$$

The $\odot$ is elementwise because σ is applied elementwise, so its Jacobian is diagonal — and multiplying by a diagonal matrix is an elementwise product. Then, identically to before:

$$\partial L/\partial W_1 = \delta_1 x^{T} \;\cdot\; \partial L/\partial b_1 = \delta_1$$

Two patterns repeat at every layer, forever: propagate backwards by the *transposed* weight matrix, and multiply elementwise by the activation's derivative.

Check the shapes

Shape errors are the most common bug in hand-written backprop, and the dimensions are forced:

Quantity	Shape	Why
$\mathbf{x}$	(n,)	input
W_1	(h, n)	maps n → h
δ_1	(h,)	one per hidden pre-activation
$\partial L/\partial W_1 = \delta_1 \mathbf{x}^T$	(h, n)	must match W_1
W_2	(1, h)	maps h → 1
δ_2	(1,)	scalar output
$W_2^T \delta_2$	(h,)	back into hidden space

A gradient always has the shape of the thing it differentiates. If yours does not, stop — you have transposed something.

Why gradients vanish

$\delta_1 = (W_2^T \delta_2) \odot \sigma'(z_1)$. Across L layers that is a product of L factors of the form $W^T \odot \sigma'$. With sigmoid, $\sigma' = \sigma(1-\sigma) \le \mathbf{0.25}$, so ten stacked sigmoids attenuate the gradient by at most $0.25^{10} \approx 10^{-6}$ from the activations alone, before the weights contribute anything.

That single line is the reason ReLU ($\sigma' = 1$ on the positive half) took over, and the reason residual connections — which add an identity path whose derivative is exactly 1 — made 100-layer networks trainable.

```
"""Backpropagation by hand, checked against numerical gradients."""

import numpy as np


def numeric_jacobian(f, x: np.ndarray, h: float = 1e-6) -> np.ndarray:
    """Approximates the Jacobian of f at x by central differences.

    Args:
        f: Function from R^n to R^m.
        x: Point of evaluation, shape (n,).
        h: Step size.

    Returns:
        The Jacobian, shape (m, n).
    """
    x = np.asarray(x, dtype=np.float64)
    base = np.atleast_1d(f(x))
    jac = np.zeros((base.size, x.size))
    for i in range(x.size):
        step = np.zeros_like(x)
        step[i] = h
        jac[:, i] = (np.atleast_1d(f(x + step)) -
                     np.atleast_1d(f(x - step))) / (2.0 * h)
    return jac


def forward_backward(w1, b1, w2, b2, x, y):
    """Runs one forward and backward pass of a two-layer tanh network.

    Args:
        w1: First weight matrix, shape (hidden, in).
        b1: First bias, shape (hidden,).
        w2: Second weight matrix, shape (1, hidden).
        b2: Second bias, shape (1,).
        x: Input vector, shape (in,).
        y: Target scalar.

    Returns:
        A tuple (loss, gradients) where gradients is a dict.
    """
    z1 = w1 @ x + b1
    a1 = np.tanh(z1)
    z2 = w2 @ a1 + b2
    loss = 0.5 * float(((z2 - y) ** 2).item())

    d2 = (z2 - y)                           # dL/dz2
    g_w2 = np.outer(d2, a1)
    g_b2 = d2
    d1 = (w2.T @ d2) * (1.0 - a1 ** 2)      # tanh'(z) = 1 - tanh²(z)
    g_w1 = np.outer(d1, x)
    g_b1 = d1
    return loss, {'w1': g_w1, 'b1': g_b1, 'w2': g_w2, 'b2': g_b2}
```

Check it numerically and the agreement should be about 1e-9. If it is not, your analytic gradient is wrong — and a wrong gradient will still train, just badly, which is why this check saves days.

7.4 Curvature and convexity

The **Hessian** is the matrix of second partials, and it is symmetric whenever the second partials are continuous (Clairaut), so Chapter 5's spectral theorem applies. At a stationary point:

Hessian	Point
positive definite (all $\lambda > 0$)	local minimum
negative definite (all $\lambda < 0$)	local maximum
indefinite (mixed signs)	**saddle**
singular (some $\lambda = 0$)	inconclusive

A function is **convex** when its Hessian is positive semi-definite everywhere, and convexity is what makes optimisation honest: every local minimum is global. Linear and logistic regression are convex; neural networks are emphatically not.

In high dimensions, though, the usual fear is misplaced. A local minimum requires *every one* of millions of eigenvalues to be positive simultaneously; if signs were even roughly independent, that is vanishingly unlikely. Critical points in large networks are therefore overwhelmingly **saddles**, which first-order methods escape by following the negative-curvature direction. This is why gradient descent works far better in practice than one-dimensional intuition predicts.

7.5 The curse of dimensionality

High-dimensional space is not big three-dimensional space, and the arithmetic is brutal *[MML p.443]*. The volume of a ball of radius r in d dimensions scales as r^d, so the fraction lying within the outer shell of thickness ε is

$$1 - (1 - \varepsilon)^{d}$$

d	ε = 0.05
2	9.8%
10	40.1%
50	92.3%
100	**99.4%**

Essentially all the volume is at the surface. Two consequences you will meet constantly:

- **Distances concentrate.** For many distributions the ratio of farthest to nearest neighbour tends to 1 as d grows, so "nearest neighbour" degrades into "any neighbour", and distance-based methods lose their meaning.
- **Sample requirements explode.** To keep a fixed density you need exponentially many points in d.

This is why dimensionality reduction (Chapter 5), regularisation (Chapter 12) and representation learning (Chapter 20) are not optional refinements. They are what make learning in high dimensions possible at all.

7.6 Gate for this chapter

Check	Threshold
Analytic vs numeric Jacobian on a 3→2 map	< 1e-6
Hand-written backprop vs numerical gradient	< 1e-7 relative
Every gradient's shape matches its parameter	exact
Shell-volume table reproduced at d = 2, 10, 50, 100	to 0.1%
Rosenbrock with and without momentum	iteration counts recorded for both

8 · Probability, information, and where losses come from

8.1 The axioms

A probability measure P assigns numbers to events with $P(A) \geq 0$, $P(\Omega) = 1$, and countable additivity for disjoint events *[MML p.498]*. Everything else is consequence.

Conditional probability $P(A|B) = P(A \cap B)/P(B)$ *[MML p.515]* rearranges into Bayes' theorem. The derivation is one line — write $P(A \cap B)$ two ways:

$$P(A|B)P(B) = P(A \cap B) = P(B|A)P(A) \implies P(A|B) = P(B|A)P(A)/P(B)$$

Worked example — the base rate, in counts

Symbols hide the result; counts reveal it. A test is 99% sensitive and 99% specific; the disease affects 1 in 1000. Imagine **100,000 people**:

	Sick (100)	Healthy (99,900)	Total
Test +	99	999	1,098
Test −	1	98,901	98,902

Of the 1,098 positives, 99 are genuinely sick:

$$P(\text{sick} \mid +) = {}^{99}/_{1098} \approx 0.090$$

About **9%**. A 99%-accurate test on a rare condition produces ten false positives for every true one, because the healthy group is a thousand times larger and even a 1% error rate on it swamps the sick group entirely.

Keep this in your pocket: it is the same arithmetic as a fraud detector with a 1% base rate, and it is why precision collapses on imbalanced data no matter how good the classifier's accuracy looks (Chapter 14).

8.2 Random variables and distributions

A random variable maps outcomes to numbers *[MML p.534]*. Discrete ones carry a probability mass function; continuous ones a density f, where probability is the integral over a region, and $P(X = x) = 0$ pointwise *[MML p.567]*.

Distribution	Use it for	Mean / Variance
Bernoulli(p)	one binary outcome	p / p(1−p)
Binomial(n,p)	successes in n trials	np / np(1−p)
Poisson(λ)	counts per interval	λ / λ
Normal(μ,σ²)	sums of many small effects	μ / σ²
Exponential(λ)	waiting times	1/λ / 1/λ²

The Normal earns its ubiquity from the **Central Limit Theorem**: for i.i.d. X_i with mean μ and finite variance σ²,

$$\sqrt{n}(\bar{X}_n - \mu) \to \text{Normal}(0, \sigma^2) \quad \text{in distribution}$$

regardless of what the X_i individually looked like. That is why measurement noise is usually modelled as Gaussian — and also why heavy-tailed data (prices, traffic, incomes), where σ² may not be finite, breaks models that assume it.

8.3 Expectation, variance, and the law of large numbers

$$\mathbb{E}[X] = \Sigma_x \, x \cdot p(x) \quad \text{or} \quad \int x \cdot f(x)\, dx \quad \cdot \quad \text{Var}(X) = \mathbb{E}[(X - \mathbb{E}[X])^2]$$

The computational form follows by expanding the square:

$$\text{Var}(X) = \mathbb{E}[X^2 - 2X\mathbb{E}[X] + \mathbb{E}[X]^2] = \mathbb{E}[X^2] - 2\mathbb{E}[X]^2 + \mathbb{E}[X]^2 = \mathbb{E}[X^2] - \mathbb{E}[X]^2$$

Expectation is linear *always* — $\mathbb{E}[aX + bY] = a\mathbb{E}[X] + b\mathbb{E}[Y]$ even for dependent variables *[MML p.587]*. Variance is not:

$$\text{Var}(X + Y) = \text{Var}(X) + \text{Var}(Y) + 2\text{Cov}(X, Y)$$

so it adds only under independence. Nearly every "my confidence interval is too narrow" bug traces back to that second sentence — correlated errors treated as independent.

Why error shrinks like 1/√n

For i.i.d. X_i, linearity gives $\mathbb{E}[\bar{X}] = \mu$, and independence gives

$$\text{Var}(\bar{X}_n) = \text{Var}((1/n)\Sigma X_i) = (1/n^2) \cdot n \cdot \sigma^2 = \sigma^2/n$$

so the standard error is σ/√n. ■ **Every estimate you report is subject to that √n**: quadrupling the data halves the error bar, and no amount of modelling cleverness changes the exponent. It is also why a single train/test split is noisy, and why Chapter 14 insists on bootstrap intervals.

Stable variance

Compute variance in one pass with Welford's algorithm rather than the textbook $\mathbb{E}[X^2] - \mathbb{E}[X]^2$, which catastrophically cancels for large means: with mean $\approx 10^9$, both terms are $\approx 10^{18}$ while their difference is ≈ 1, and double precision carries only ~16 digits — so the answer is noise.

```
"""Welford's online mean and variance."""

import numpy as np


def welford(values: np.ndarray) -> tuple:
    """Computes mean and sample variance in one numerically stable pass.

    Args:
        values: One-dimensional array of observations.

    Returns:
        A tuple (mean, sample_variance).
    """
    count = 0
    mean = 0.0
    m2 = 0.0
    for value in np.asarray(values, dtype=np.float64).ravel():
        count += 1
        delta = value - mean
        mean += delta / count
        m2 += delta * (value - mean)
    return mean, m2 / (count - 1) if count > 1 else 0.0


big = np.float64(1e9) + np.random.default_rng(0).normal(size=100_000)
naive = float((big ** 2).mean() - big.mean() ** 2)   # cancels catastrophically
print('naive variance  :', naive)
print('welford variance:', welford(big)[1])
print('numpy variance  :', float(big.var(ddof=1)))
```

8.4 Information: entropy, cross-entropy, KL

Entropy measures the average surprise in a distribution *[MML p.606]*:

$$H(p) = -\Sigma_x\, p(x) \cdot \log\, p(x)$$

Worked example — four outcomes

Distribution	Computation	Entropy (bits)
(¼,¼,¼,¼)	$-4 \cdot \frac{1}{4} \cdot \log_2 \frac{1}{4}$	**2.000**
(½,¼,⅛,⅛)	$-(\frac{1}{2} \cdot -1 + \frac{1}{4} \cdot -2 + 2 \cdot \frac{1}{8} \cdot -3)$	**1.750**
(1,0,0,0)	0	**0.000**

Uniform is maximal ($\log_2 4 = 2$ bits), certainty is zero, and the skewed case sits between. In bits, entropy is literally the optimal average code length — the second distribution really can be coded in 1.75 bits per symbol on average, and no scheme does better.

Cross-entropy is the cost of coding p with a code built for q:

$$H(p, q) = -\Sigma_x \, p(x) \cdot \log q(x)$$

and the excess you pay for being wrong is the **Kullback–Leibler divergence**:

$$D_{KL}(p \parallel q) = \Sigma_x \, p(x) \cdot \log(p(x) \,/\, q(x)) = H(p, q) - H(p)$$

Why KL is never negative

This is Gibbs' inequality, and it follows from the concavity of the logarithm via **Jensen's inequality** ($\mathbb{E}[\varphi(X)] \le \varphi(\mathbb{E}[X])$ for concave φ):

$$-D_{KL}(p\|q) = \Sigma_x \, p(x) \cdot \log(q(x)/p(x)) \le \log \Sigma_x \, p(x) \cdot (q(x)/p(x)) = \log \Sigma_x \, q(x) = \log 1 = 0$$

so $D_{KL} \ge 0$, with equality exactly when $q(x)/p(x)$ is constant — that is, when $p = q$. ■

KL is also **not symmetric**: $D_{KL}(p\|q) \ne D_{KL}(q\|p)$. Take $p = (0.5, 0.5)$ and $q = (0.9, 0.1)$:

$$D_{KL}(p\|q) = 0.5 \cdot \log(0.5/0.9) + 0.5 \cdot \log(0.5/0.1) \approx 0.511 \text{ nats}$$

$$D_{KL}(q\|p) = 0.9 \cdot \log(0.9/0.5) + 0.1 \cdot \log(0.1/0.5) \approx 0.368 \text{ nats}$$

Different numbers — which is why it is a divergence rather than a distance, and why the *direction* matters when you fit a VAE (Chapter 20). The forward direction punishes q for putting zero mass where p has some (mode-covering); the reverse punishes q for spreading mass where p has none (mode-seeking).

Now the payoff. Since $H(p)$ does not depend on your model, **minimising cross-entropy is exactly minimising KL divergence to the truth**. The loss function in every classifier you will ever train is a statement about information.

8.5 Maximum likelihood: where every loss comes from

Given data D and a parametric model, the **likelihood** is P(D|θ), and maximum likelihood picks the θ making the observed data most probable *[MML p.619]*. Products underflow, so work with the log — and since log is monotone, the maximiser is unchanged:

$$\hat{\theta} = \operatorname{argmax}_{\theta} \Sigma_i \log p(x_i \mid \theta)$$

Derivation 1 — Gaussian noise gives squared error

Assume $y_i = f(x_i;\theta) + \varepsilon_i$ with $\varepsilon \sim N(0, \sigma^2)$. The density of one observation is

$$p(y_i|x_i,\theta) = (1/(\sigma\sqrt{(2\pi)}))\cdot\exp(-(y_i - f(x_i;\theta))^2/(2\sigma^2))$$

Take logs and sum:

$$\log L = -(\tfrac{1}{2}\sigma^2)\cdot\Sigma_i\,(y_i - f(x_i;\theta))^2 - n\cdot\log\sigma - (n/2)\cdot\log(2\pi)$$

The last two terms are constant in θ. Maximising log L is therefore *minimising* $\Sigma(y_i - f(x_i))^2$ — **least squares**. ■

Least squares was never an arbitrary choice; it is the MLE under Gaussian noise. That is also the honest reason to distrust MSE on heavy-tailed targets: the *assumption* fails, not the arithmetic. Assume Laplace noise instead and the same derivation yields **MAE**.

Derivation 2 — Bernoulli outcomes give cross-entropy

With $p_i = f(x_i;\theta) \in (0,1)$ and $y_i \in \{0,1\}$, a single observation has probability $p_i^{y_i}(1-p_i)^{1-y_i}$ — which equals p_i when $y_i=1$ and $1-p_i$ when $y_i=0$. Logging:

$$\log L = \Sigma_i\,[\, y_i\cdot\log p_i + (1 - y_i)\cdot\log(1 - p_i)\,]$$

Negate and you have exactly the **binary cross-entropy** of *[PTG p.39]*. The categorical case gives multi-class cross-entropy identically. ■

So the two losses that dominate machine learning are not conventions. They are maximum-likelihood estimates under two noise models, and choosing a loss is choosing a probabilistic assumption about your data — whether you realise it or not.

Why the MLE variance is biased

Maximising over σ^2 too gives $\hat{\sigma}^2 = (1/n)\Sigma(x_i - \hat{\mu})^2$. But

$$\mathbb{E}[\hat{\sigma}^2] = ((n-1)/n)\cdot\sigma^2 < \sigma^2$$

because the same data was used to estimate μ, and the residuals are measured from the sample mean, which sits closer to the data than the true mean does. Correcting the factor gives the familiar 1/(n−1). **The MLE is not always unbiased** — a useful antidote to treating it as automatically correct.

```
"""Entropy, KL, and maximum likelihood estimates."""

import numpy as np


def entropy(p: np.ndarray, base: float = np.e) -> float:
    """Shannon entropy of a probability vector.

    Args:
        p: Probabilities, summing to 1.
        base: Logarithm base; e gives nats, 2 gives bits.

    Returns:
        The entropy; zero probabilities contribute zero.

    Raises:
        ValueError: If p does not sum to 1 or has negative entries.
    """
    p = np.asarray(p, dtype=np.float64)
    if np.any(p < 0) or abs(p.sum() - 1.0) > 1e-9:
        raise ValueError('p must be a probability vector')
    nonzero = p[p > 0]
    return float(-np.sum(nonzero * np.log(nonzero)) / np.log(base))


def kl_divergence(p: np.ndarray, q: np.ndarray) -> float:
    """Kullback-Leibler divergence from q to p, in nats.

    Args:
        p: Reference distribution.
        q: Comparison distribution.

    Returns:
        The divergence, always >= 0.

    Raises:
        ValueError: If q is zero where p is positive.
    """
    p = np.asarray(p, dtype=np.float64)
    q = np.asarray(q, dtype=np.float64)
    mask = p > 0
    if np.any(q[mask] <= 0):
        raise ValueError('KL is infinite where q = 0 but p > 0')
    return float(np.sum(p[mask] * np.log(p[mask] / q[mask])))


def mle_gaussian(samples: np.ndarray) -> tuple:
    """Maximum-likelihood mean and variance of a Gaussian.

    Note the 1/n divisor: the MLE of the variance is biased, which is
    why the unbiased estimator uses 1/(n-1) instead.

    Args:
        samples: Observed values.

    Returns:
        A tuple (mu_hat, sigma_squared_hat).
    """
    x = np.asarray(samples, dtype=np.float64)
    mu = float(x.mean())
    return mu, float(((x - mu) ** 2).mean())
```

8.6 Gate for this chapter

Check	Threshold
Entropy of uniform over n	log n, to 1e-12
Your hand-computed entropies (2.000, 1.750, 0.000 bits)	exact
D_KL(p‖p) = 0 and D_KL ≥ 0 over 1000 random pairs	exact / non-negative
Asymmetry example (0.511 vs 0.368 nats)	to 1e-3
MLE recovers μ, σ² from 1e5 samples	within 3 standard errors
Welford vs naive variance at mean 1e9	naive visibly wrong
Bayes worked example	0.090 ± 0.001

Part II — Learning from Data

Part I built the engine. Part II is about everything that decides whether a model is *true*: how the problem is framed, how the data is split, and how the result is measured. Almost every failure in applied machine learning happens here rather than in the model.

9 · Framing the problem

9.1 Decide the metric before the first fit

The most expensive mistakes are made before any code runs. Write these down, in this order, *before* fitting anything *[SKL ch.1 "Choosing Metrics and Thresholds That Match with the Goal"]*:

1. **The decision.** What will change because of this prediction? If nothing changes, stop.
2. **The cost of being wrong**, in both directions. A missed fraud and a blocked legitimate customer are not the same cost, and the ratio between them determines the threshold.
3. **The target.** Exactly which quantity, measured when, available at prediction time. ("Available at prediction time" is doing more work in that sentence than any other phrase in this book — see Chapter 10.)
4. **The metric**, chosen from the cost above, not from habit.
5. **The baseline.** Predict the mean, the majority class, or last week's value. If you cannot beat it, you have learned something important and cheap.

The reason for the ordering is brutal: once you have seen a score, you cannot un-see it, and every subsequent choice of metric is contaminated by knowing which one flatters you.

Worked example — the threshold follows from the costs

Suppose a false negative (missed fraud) costs £500 and a false positive (a blocked good customer) costs £5. Expected cost of alarming at probability p:

$$\text{alarm: } 5\cdot(1-p) \quad \cdot \quad \text{stay silent: } 500\cdot p$$

Alarm when $5(1-p) < 500p$, i.e. $5 < 505p$, i.e. $\mathbf{p > 0.0099}$. The optimal threshold is 1%, not 50%. In general:

$$p^* = C_{FP} / (C_{FP} + C_{FN})$$

The default 0.5 threshold is the special case where both errors cost the same — which is almost never true, and is why the threshold is a *business* parameter you tune on validation data, never a property of the model.

9.2 Problem types

Type	Target	Typical metric
Binary classification	one of two labels	PR-AUC, F1, recall at fixed precision
Multi-class	one of k	macro-F1, balanced accuracy
Multi-label	any subset of k	per-label F1, subset accuracy
Regression	a real number	MAE, RMSE, R^2
Forecasting	a future sequence	MASE, RMSSE (Chapter 23)
Ranking	an ordering	NDCG, MAP
Clustering	no target at all	silhouette *plus* stability

9.3 Schema before model

Type every column before modelling *[SKL ch.3 "Foundations: Why Preprocessing Matters and How to Read Data as a Schema"]*. Not the dtype pandas guessed — the *logical* type, which determines the treatment:

Logical type	Treatment
Continuous	scale; consider a transform if skewed
Count	often log1p; consider Poisson-style losses
Ordinal	integer encoding, order preserved
Nominal	one-hot, or target encoding fitted inside the fold
Datetime	decompose into cyclical parts; never feed a raw timestamp
Text	TF-IDF, then reduce (Chapter 15)
Identifier	**drop it** — and check it is not leaking (Chapter 10)

A column whose logical type you cannot state is a column you do not understand, and it will bite you later.

10 · Leakage

> Leakage is the use, at training time, of information that will not exist at prediction time. It does not announce itself. It appears as an excellent score.

This chapter takes its taxonomy from *[TSP p.158–p.164]*, the most rigorous treatment in the sources, and applies it to *all* work, temporal or not.

10.1 The demonstration

Here is leakage with nothing else in the picture. The features are pure noise; the target is independent of them. **There is no signal.** Any positive score is manufactured.

```
"""Feature selection outside the fold invents signal from pure noise."""

import numpy as np


def fit_predict(x_train, y_train, x_test):
    """Least-squares fit with an intercept, used as a neutral probe."""
    design = np.hstack([x_train, np.ones((x_train.shape[0], 1))])
    weights, *_ = np.linalg.lstsq(design, y_train, rcond=None)
    test_design = np.hstack([x_test, np.ones((x_test.shape[0], 1))])
    return test_design @ weights


def r2(y_true, y_pred):
    """Coefficient of determination."""
    total = float(np.sum((y_true - y_true.mean()) ** 2))
    return 1.0 - float(np.sum((y_true - y_pred) ** 2)) / total


rng = np.random.default_rng(20260101)
n, p, keep = 200, 2000, 20
x = rng.normal(size=(n, p))
y = rng.normal(size=n)                      # independent of x. No signal.

idx = rng.permutation(n)
train, test = idx[:140], idx[140:]

# LEAKING: rank features using every row, including the test rows.
leaked = np.argsort(np.abs(x.T @ (y - y.mean())))[-keep:]
leaked_r2 = r2(y[test], fit_predict(x[train][:, leaked], y[train],
                                    x[test][:, leaked]))

# HONEST: rank features using the training rows only.
honest = np.argsort(np.abs(x[train].T @ (y[train] - y[train].mean())))[-keep:]
honest_r2 = r2(y[test], fit_predict(x[train][:, honest], y[train],
                                    x[test][:, honest]))

print(f'leaked R² = {leaked_r2:+.3f}   honest R² = {honest_r2:+.3f}')
```

On the seed above this prints roughly **leaked R² = +0.35, honest R² = −0.39**. A third of the variance "explained" in data that contains none.

Why the inflation is exactly this large

The mechanism is quantifiable, which makes it much harder to dismiss. With p independent noise features, the sample correlation of each with the target is roughly N(0, 1/n). Selecting the top 20 of 2,000 is selecting the extreme upper tail of 2,000 draws — and the maximum of p standard normals grows like √(2 log p), which for p = 2000 is about 3.9 standard deviations.

Those features look predictive **on the rows used to select them**, which here include the test rows. The test set is no longer independent of the model — it participated in choosing the features — so it cannot measure generalisation. The honest version selects on training rows only; the chosen features look good there and, correctly, mean nothing on the test rows, giving the slightly negative R^2 you would expect from fitting noise.

Run this before you trust any pipeline you have built. It is the cheapest inoculation in this book.

10.2 The taxonomy

Kind	Mechanism	Anchor
Transformation	A scaler, imputer or encoder fitted on all rows before splitting. Test statistics flow into training.	*[TSP p.159]*
Selection	Features chosen using the full dataset, as above.	*[SKL ch.9]*
Lookforward	Any use of future information: a shuffled split on a time series, a rolling statistic over the whole series, a label that post-dates the prediction.	*[TSP p.161]*
Panel / group	The same entity in both train and test — a patient, a customer, a machine. The model memorises the entity, not the pattern.	*[TSP p.163]*
Target-derived	A feature computed from the target, usually in disguise: a group aggregate, a target encoding fitted outside the fold.	*[TSP p.159]*
Duplicate rows	Near-duplicates straddling the split.	*[SKL ch.3]*

10.3 The checklist

From *[TSP p.164]*, in the order that matters:

1. **Split first.** Before any transformation. Always.
2. Fit every parameterised transform — scalers, imputers, encoders, PCA, selectors — **inside** the fold, on training rows only.
3. Respect chronology where time exists.
4. Keep entities whole: group-aware splits for panel data.
5. For every feature, ask *when would this value be known?* If the answer is "after the thing I am predicting", drop it.

6. Keep the test set sealed until the very end — one look, one number.
7. Document the pipeline so a reviewer can check 1–6 without reading code.

10.4 The one defence that scales

Rules you must remember are rules you will eventually forget. Structure beats vigilance: put the entire transformation chain inside a single estimator object and let the cross-validation machinery do the splitting *[SKL ch.3 "Putting It Together: One Leak-Proof ColumnTransformer"]*. Then "fit on train only" is not a discipline you maintain — it is what the code does.

---

11 · Pipelines that cannot leak

11.1 The shape

```python
"""One leak-proof estimator: preprocessing and model as a single object."""

from sklearn.compose import ColumnTransformer
from sklearn.ensemble import HistGradientBoostingClassifier
from sklearn.impute import SimpleImputer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler


def build_pipeline(numeric: list, categorical: list) -> Pipeline:
    """Builds a leak-proof preprocessing and modelling pipeline.

    Every parameterised step lives inside the pipeline, so it is fitted
    on training folds only when the pipeline is cross-validated.

    Args:
        numeric: Names of the continuous columns.
        categorical: Names of the nominal columns.

    Returns:
        An unfitted scikit-learn Pipeline.
    """
    numeric_steps = Pipeline([
        ('impute', SimpleImputer(strategy='median', add_indicator=True)),
        ('scale', StandardScaler()),
    ])
    categorical_steps = Pipeline([
        ('impute', SimpleImputer(strategy='most_frequent')),
        ('encode', OneHotEncoder(handle_unknown='ignore', min_frequency=10)),
    ])
    preprocess = ColumnTransformer([
        ('num', numeric_steps, numeric),
        ('cat', categorical_steps, categorical),
    ], remainder='drop')
    return Pipeline([
        ('prep', preprocess),
        ('model', HistGradientBoostingClassifier(random_state=0)),
    ])
```

Three details carry most of the value. `handle_unknown='ignore'` stops an unseen category from crashing inference. `add_indicator=True` keeps the fact that a value *was* missing, which is often more predictive than whatever you impute. `remainder='drop'` means a new column cannot silently join the model without you naming it.

11.2 Prove it

Do not take the pipeline's word for it. Measure the gap:

```python
from sklearn.model_selection import cross_val_score

honest = cross_val_score(build_pipeline(num_cols, cat_cols), X, y,
                         cv=5, scoring='average_precision')

# The wrong way, for contrast: transform everything up front, then CV.
X_all = preprocess.fit_transform(X)          # sees every row
leaky = cross_val_score(model, X_all, y, cv=5, scoring='average_precision')

print(f'honest {honest.mean():.4f}  leaky {leaky.mean():.4f}  '
      f'inflation {leaky.mean() - honest.mean():+.4f}')
```

Record that inflation number. It is the size of the lie you would have shipped.

11.3 Splitters

The pipeline protects you from transformation leakage. It does **not** protect you from the other kinds — those are the splitter's job *[SKL ch.3 "Leakage-Safe Evaluation"]*:

Data	Splitter
i.i.d. rows	`KFold`, or `StratifiedKFold` when classes are imbalanced
Repeated entities	`GroupKFold`, `StratifiedGroupKFold`
Time	`TimeSeriesSplit`, or the rolling/expanding designs of Chapter 23
Both group and time	a custom splitter; write the boundaries out and assert them

12 · Linear models

12.1 The bias–variance decomposition

Before any particular model, the result that explains why regularisation can *improve* predictions despite making the fit worse. Let $y = f(x) + \varepsilon$ with $\mathbb{E}[\varepsilon] = 0$, $\mathrm{Var}(\varepsilon) = \sigma^2$, and let $\hat{f}$ be our estimate (random, because the training set is random). The expected squared error at a point x is

$$\mathbb{E}[(y - \hat{f}(x))^2] = (f(x) - \mathbb{E}[\hat{f}(x)])^2 + \mathbb{E}[(\hat{f}(x) - \mathbb{E}[\hat{f}(x)])^2] + \sigma^2$$

Derivation. Write $y - \hat{f} = (y - f) + (f - \mathbb{E}[\hat{f}]) + (\mathbb{E}[\hat{f}] - \hat{f})$. Square and take expectations. The cross terms vanish: ε is independent of the training data and has zero mean, and $\mathbb{E}[\mathbb{E}[\hat{f}] - \hat{f}] = 0$ by construction. What survives is the three terms above. ■

$$\text{Error} = \text{Bias}^2 + \text{Variance} + \text{Irreducible noise}$$

Three consequences worth internalising:

- **σ^2 is a floor.** No model beats it; a reported error below it means a leak (Chapter 10) or a bug.
- **Bias and variance trade.** More capacity lowers bias and raises variance.
- **Regularisation buys variance reduction with bias.** It is a *deliberately worse fit* that generalises better — which is exactly what ridge does next.

12.2 Ridge, derived — and what α really does

Ordinary least squares minimises $\|X\mathbf{w} - \mathbf{y}\|^2$. Ridge adds a penalty:

$$L(w) = \|Xw - y\|^2 + \alpha\|w\|^2$$

Differentiate — the same move as Chapter 4, with one extra term:

$$\nabla L = 2X^T(Xw - y) + 2\alpha w = 0 \implies (X^TX + \alpha I)w = X^Ty$$

In the SVD basis, ridge becomes transparent

Write $X = U\Sigma V^T$ (Chapter 5). Substituting and simplifying:

$$w_{\text{ridge}} = \Sigma_i \left[\sigma_i / (\sigma_i^2 + \alpha) \right] \langle u_i, y \rangle v_i$$

whereas OLS has $1/\sigma_i$ in place of $\sigma_i/(\sigma_i^2 + \alpha)$. So ridge multiplies each component by a **shrinkage factor**

$$\sigma_i^2 / (\sigma_i^2 + \alpha) \in (0, 1)$$

Now read off the behaviour:

Direction	Shrinkage factor	Effect
$\sigma_i^2 \gg \alpha$ (strong, well-determined)	≈ 1	barely touched
$\sigma_i^2 \approx \alpha$	0.5	halved
$\sigma_i^2 \ll \alpha$ (weak, noise-dominated)	≈ 0	suppressed

Ridge does not shrink everything equally — it shrinks the directions your data barely determines. Those are precisely the directions where OLS divides by a tiny σ_i and amplifies noise, which is the $\kappa(X)$ catastrophe of Chapter 4 appearing in statistical dress. The condition number of the ridge system is

$$\kappa = (\sigma_{max}^2 + \alpha)/(\sigma_{min}^2 + \alpha) < \kappa(X^TX)$$

so **regularisation is conditioning**, and the statistical story (variance reduction) and the numerical story (stability) are the same story.

```python
import numpy as np


def ridge_fit(x: np.ndarray, y: np.ndarray, alpha: float) -> np.ndarray:
    """Fits ridge regression in closed form, leaving the intercept free.

    Args:
        x: Feature matrix of shape (n, d).
        y: Targets of shape (n,).
        alpha: Non-negative penalty strength.

    Returns:
        Coefficients of shape (d + 1,), intercept last.

    Raises:
        ValueError: If alpha is negative.
    """
    if alpha < 0:
        raise ValueError('alpha must be non-negative')
    x_mean, y_mean = x.mean(axis=0), y.mean()
    xc, yc = x - x_mean, y - y_mean
    d = x.shape[1]
    w = np.linalg.solve(xc.T @ xc + alpha * np.eye(d), xc.T @ yc)
    return np.append(w, y_mean - x_mean @ w)
```

The intercept is excluded from the penalty deliberately: shrinking it would tie predictions to the arbitrary location of your coordinate system.

12.3 L1 versus L2, finished properly

Lasso penalises $\|\mathbf{w}\|_1$. Chapter 2 noted the L1 ball has corners on the axes; here is why that produces exact zeros, in one dimension where it is provable by hand.

Minimise $\tfrac{1}{2}(w - c)^2 + \alpha|w|$ — a least-squares term pulling w towards c, and an L1 penalty pulling towards zero. For $w > 0$ the derivative is $(w - c) + \alpha$, zero at $w = c - \alpha$. For $w < 0$ it is $(w - c) - \alpha$, zero at $w = c + \alpha$. And at $w = 0$ the subgradient is $[-\alpha, \alpha]$, so zero is optimal whenever $|c| \le \alpha$. The solution is the **soft-threshold**:

$$w^* = \operatorname{sign}(c) \cdot \max(|c| - \alpha, 0)$$

Exactly zero whenever $|c| \le \alpha$ — a genuine selection, not a small number.

Now L2: minimise $\tfrac{1}{2}(w - c)^2 + \alpha w^2$. Differentiating gives $w - c + 2\alpha w = 0$, so

$$w^* = c \,/\, (1 + 2\alpha)$$

which is zero only when c is. L2 shrinks proportionally and never selects.

> **L1 selects, L2 shrinks** — and now you know it is a statement about the subgradient at the origin, not folklore.

12.4 Logistic regression

For binary targets, model the log-odds as linear and squash with the sigmoid:

$$p(x) = \sigma(w^{\mathrm{T}}x + b), \quad \sigma(z) = 1 \,/\, (1 + e^{-z})$$

Fit by maximum likelihood — which Chapter 8 showed is minimising binary cross-entropy.

The gradient, derived

Two facts make this short. First, $\sigma'(z) = \sigma(z)(1 - \sigma(z))$ — differentiate (1 + e^{−z})^{−1} and regroup. Second, the per-sample loss is $-[y \log p + (1-y) \log(1-p)]$. Differentiating with respect to z:

$$\partial L/\partial z = -[\, y/p - (1-y)/(1-p) \,] \cdot p(1-p) = -[\, y(1-p) - (1-y)p \,] = p - y$$

The σ' factor **cancels exactly** against the derivative of the log. Then by the chain rule, since $z = \mathbf{w}^{\mathrm{T}}\mathbf{x}$:

$$\partial L/\partial w = X^{\mathrm{T}}(p - y)$$

Prediction minus truth, projected back through the features. ■

The identical form appears in linear regression with squared error, and in the softmax layer of Chapter 17. Not a coincidence: it is what happens whenever the loss is the negative log-likelihood of an exponential-family distribution with its matching link function.

Why it is convex, and what Newton does

The Hessian is

$$H = X^T S X, \quad S = \mathrm{diag}(p_i(1 - p_i))$$

Every $p_i(1-p_i) > 0$, so for any $\mathbf{v}$, $\mathbf{v}^T H \mathbf{v} = \|S^{1/2} X \mathbf{v}\|^2 \geq 0$. **The Hessian is positive semi-definite everywhere, so the problem is convex** and every local optimum is global — unlike anything in Part III.

Newton's method then gives the update

$$w \leftarrow w + (X^T S X)^{-1} X^T (y - p)$$

which is a weighted least-squares solve — hence the classical name **iteratively reweighted least squares**, and the reason logistic regression converges in a handful of iterations rather than the thousands gradient descent would need.

Logistic regression is the strongest baseline in applied ML: fast, calibrated, interpretable, convex. Beat it before you reach for anything deeper.

---

13 · Trees and ensembles

13.1 One tree, one split

A decision tree splits the feature space with axis-aligned cuts, choosing at each node the split that most reduces impurity:

$$\text{Gini} = 1 - \Sigma_k \, p_k{}^2 \quad \cdot \quad \text{Entropy} = -\Sigma_k \, p_k \log p_k$$

That entropy is Chapter 8's entropy, and the "information gain" of a split is the reduction in it.

Worked example — choosing between two splits

Parent node: 100 samples, 50 positive. Gini = $1 - (0.5^2 + 0.5^2) =$ **0.5**.

Split A → left (50: 40 pos), right (50: 10 pos):

$$G_{\text{left}} = 1 - (0.8^2 + 0.2^2) = 0.32 \quad \cdot \quad G_{\text{right}} = 1 - (0.2^2 + 0.8^2) = 0.32$$

$$\text{weighted} = 0.5 \cdot 0.32 + 0.5 \cdot 0.32 = 0.32 \implies \text{gain} = 0.5 - 0.32 = \mathbf{0.18}$$

Split B → left (10: 10 pos), right (90: 40 pos):

$$G_{\text{left}} = 0 \quad \cdot \quad G_{\text{right}} = 1 - ((40/90)^2 + (50/90)^2) \approx 0.494$$

$$\text{weighted} = 0.1 \cdot 0 + 0.9 \cdot 0.494 \approx 0.444 \implies \text{gain} \approx \mathbf{0.056}$$

Split A wins, despite Split B producing one perfectly pure leaf — because that leaf is small, and the weighting by node size is what stops trees from chasing tiny pure corners. Note also that the gain is **always non-negative**: a split cannot increase weighted impurity, which is exactly why an unconstrained tree grows until every leaf is pure, memorising the training set.

The depth, minimum-leaf-size and minimum-impurity-decrease parameters exist entirely to stop that.

13.2 Bagging and boosting

	Random forest	Gradient boosting
Trees are	independent, parallel	sequential, each fixing the last
Each tree is	deep, low-bias, high-variance	shallow, high-bias
Combination	average	weighted sum
Reduces	variance	bias
More trees	never hurts	eventually overfits
Tuning	forgiving	repays care

Why averaging works, and why correlation limits it

Average B estimators each with variance σ^2 and pairwise correlation ρ:

$$\text{Var}(\text{mean}) = \rho\sigma^2 + (1 - \rho)\sigma^2/B$$

As $B \to \infty$ the second term vanishes but the first does not. **The floor is $\rho\sigma^2$.** Adding trees past a point buys nothing; the only way lower is to *decorrelate* them — which is precisely what a random forest's random feature subset at each split is for. Bagging alone (bootstrap samples, all features) leaves highly correlated trees, and this formula is why Breiman added the feature sampling.

Boosting instead fits each new tree to the *negative gradient of the loss* with respect to the current predictions — Chapter 6's gradient descent, performed in function space rather than parameter space. With squared error the negative gradient is just the residual, which is why the classic description is "fit the next tree to the residuals"; with other losses it is not the residual, and the gradient view is the one that generalises.

For tabular data, gradient boosting is the honest default, and `HistGradientBoostingClassifier` is fast enough to be the first thing you try. Compare at *matched budgets*, though: a well-tuned forest beating a badly tuned booster teaches nothing.

14 · Evaluation

14.1 The confusion matrix, worked

$$\text{Precision} = \text{TP}/(\text{TP}+\text{FP}) \;\cdot\; \text{Recall} = \text{TP}/(\text{TP}+\text{FN}) \;\cdot\; \text{F1} = 2\cdot\text{P}\cdot\text{R}/(\text{P}+\text{R})$$

Take 10,000 samples with a 1% positive rate, and a model that catches 80 of the 100 positives while raising 400 false alarms:

	Predicted +	**Predicted –**
Actually +	TP = 80	FN = 20
Actually –	FP = 400	TN = 9,500

$$\text{Precision} = {}^{80}/_{480} = 0.167 \;\cdot\; \text{Recall} = {}^{80}/_{100} = 0.80 \;\cdot\; \text{F1} = 0.276$$

$$\text{Accuracy} = (80 + 9500)/10000 = 0.958$$

95.8% accuracy, and five out of six alarms are wrong. Meanwhile the do-nothing model that predicts "negative" always scores 99% accuracy. This is Chapter 8's base-rate arithmetic wearing a different hat, and it is why accuracy is banned from imbalanced reports.

Note too that F1 is the *harmonic* mean, which is why it sits much closer to the smaller of precision and recall than the arithmetic mean would — it refuses to let a good recall hide a terrible precision.

PR-AUC versus ROC-AUC

ROC plots recall against the false-positive rate FP/(FP+TN). In the table above, FPR = 400/9900 = 0.040 — a reassuring-looking number produced by 400 wrong alarms, because the denominator is the vast negative class. Precision uses FP/(TP+FP) instead, where the positive class is in both terms, and so it *feels* those 400.

Rule: when positives are rare, precision-recall is the honest curve *[SKL ch.6]*. Report both; state which one decides.

Macro versus micro

Macro averages the per-class scores (every class equal); micro pools the counts (every *sample* equal). On imbalanced data they differ sharply — so return both, always, or the reader cannot know which question you answered.

14.2 Regression metrics

$$\mathrm{MAE} = (1/n)\Sigma|y_i - \hat{y}_i| \;\cdot\; \mathrm{RMSE} = \sqrt{((1/n)\Sigma(y_i - \hat{y}_i)^2)} \;\cdot\; R^2 = 1 - SS_{res}/SS_{tot}$$

They are minimised by *different predictions*, and this is not a stylistic matter. Minimising $\mathbb{E}|y - c|$ over c gives the **median**; minimising $\mathbb{E}(y - c)^2$ gives the **mean**. So choosing MAE or RMSE is choosing what you want the model to predict, and on skewed targets those differ substantially.

MAE ≤ RMSE always (Jensen), and the **gap between them is a free diagnostic** for how heavy your error tail is: RMSE ≈ MAE means uniform errors; RMSE ≫ MAE means a few large ones dominating.

Percentage errors (MAPE) are undefined at zero targets and are not scale invariant; Chapter 23 shows why, and what to use instead.

14.3 Cross-validation and the optimism of selection

Pattern	Use
k-fold	i.i.d. data; k = 5 or 10
Stratified k-fold	imbalanced classification — the default
Group k-fold	repeated entities
Time-series split	anything temporal (Chapter 23)
Repeated k-fold	small data, to average away split noise
Nested CV	when you tune *and* report

Nested CV deserves its own argument *[SKL ch.6 "Nested CV"]*. If you select hyperparameters on the same folds you report, the reported number is optimistically biased.

Quantify it. Suppose you evaluate m candidate configurations, all genuinely equivalent, each measured with noise of standard deviation s. You report the best. The expected maximum of m draws from $N(0, s^2)$ is approximately

$$s\cdot\sqrt{(2 \log m)}$$

so with m = 100 configurations and a CV standard deviation of s = 0.02, the selection bias alone is about 0.02 × 3.03 ≈ **0.06** — six points of AUC conjured from nothing. Nested CV puts the search in an inner loop and the evaluation in an outer one, so the reported score never sees the selection. The outer score is nearly always worse, and the gap *is* your optimism. Measure it once; it will change how you read leaderboards forever.

14.4 Every number carries an interval

A single test score is a point estimate from a finite sample, with the √n behaviour of Chapter 8. Report it alone and you invite the reader to believe differences that are noise.

```python
import numpy as np


def bootstrap_ci(metric_fn, y_true, y_pred, n_boot=1000, level=0.95, seed=0):
    """Percentile bootstrap confidence interval for any metric.

    Args:
        metric_fn: Callable taking (y_true, y_pred) and returning a float.
        y_true: Ground-truth values.
        y_pred: Predicted values.
        n_boot: Number of resamples.
        level: Confidence level.
        seed: Seed for reproducibility.

    Returns:
        A (lower, upper) tuple.
    """
    rng = np.random.default_rng(seed)
    n = len(y_true)
    samples = np.empty(n_boot)
    for i in range(n_boot):
        idx = rng.integers(0, n, size=n)
        samples[i] = metric_fn(y_true[idx], y_pred[idx])
    tail = (1.0 - level) / 2.0
    return (float(np.quantile(samples, tail)),
            float(np.quantile(samples, 1.0 - tail)))
```

The bootstrap works because the empirical distribution is a consistent estimate of the true one, so resampling *from your sample* mimics resampling from the population. It needs no distributional assumption, which is why it applies to any metric — including ones with no closed-form standard error, like AUC.

A useful sanity figure: for a proportion near 0.5, the 95% interval is about ±1/√n. With n = 400 that is ±5 points. **If two models differ by less than that, you have not shown anything.**

14.5 Calibration

A model that outputs 0.8 should be right 80% of the time at that score. Many are not: boosted trees are typically over-confident, SVMs badly scaled. If any *decision* uses the probability — expected-value calculations like Section 9.1's threshold, triage, risk — calibration matters more than ranking quality.

Diagnose with a reliability curve (bin the predictions, plot mean predicted against observed frequency) and summarise with the **Brier score**, the mean squared error of the probabilities. It decomposes as

$$\text{Brier} = \text{calibration error} - \text{resolution} + \text{uncertainty}$$

which is why it rewards being both sharp *and* honest. Fix with Platt scaling (a logistic fit on held-out scores) or isotonic regression (more flexible, needs more data). Calibrate on data the model did not train

on, or you have simply moved the leak.

15 · Unsupervised structure

15.1 PCA in practice

Chapter 5 derived PCA three ways. What matters in use:

- **Standardise first** unless units are genuinely comparable — the variance being maximised has units.
- Choose k from the explained-variance curve, then **validate k downstream**: the right number of components is the one that helps the task, not the one at a visual elbow.
- Components are directions, not features with meaning, and the sign is arbitrary — never attach a story to it.
- PCA is linear. If the structure is curved it will miss it; kernel PCA or an autoencoder (Chapter 20) will not.

t-SNE and UMAP are visualisation, not features *[SKL ch.5]*. They do not preserve global distances, their output changes with perplexity and seed, and feeding their coordinates into a model is a documented error.

15.2 k-means, and why it always converges

k-means minimises within-cluster squared distance:

$$J = \Sigma_k \Sigma_{x \in C_k} \|x - \mu_k\|^2$$

alternating two steps. Both *provably* reduce J, which is the whole convergence argument:

Assignment step. Each point moves to its nearest centroid. Since each point's contribution is minimised independently and no other term changes, J cannot increase.

Update step. Each centroid moves to its cluster's mean. For fixed membership, J is a sum of $\|\mathbf{x} - \boldsymbol{\mu}\|^2$ terms whose minimiser over $\boldsymbol{\mu}$ is the mean — differentiate: $\Sigma 2(\boldsymbol{\mu} - \mathbf{x}_i) = 0$ gives $\boldsymbol{\mu}$ = mean. So again J cannot increase.

J is non-increasing and bounded below by 0, and there are finitely many possible assignments, so the algorithm terminates. ■

It never guarantees the *global* optimum (hence `n_init`), and it carries three assumptions you agree to whether you notice or not: clusters are spherical, similarly sized, and separated by Euclidean distance. Violate any one and the output will still look like clusters — that is the danger.

15.3 DBSCAN

Density-based: a point is *core* if it has `min_samples` neighbours within `eps`; clusters grow from core points; the rest is noise. It finds arbitrary shapes, decides k for itself, and reports outliers explicitly. The price is sensitivity to `eps` and failure when cluster densities differ. Pick `eps` from a k-distance plot (sort each point's distance to its k-th neighbour; take the elbow) rather than by guessing.

15.4 How to know if a clustering is real

The **silhouette** of a point compares its mean distance a to its own cluster with the mean distance b to the nearest other cluster:

$$s = (b - a) \,/\, \max(a, b) \in [-1, 1]$$

Near 1 means well-placed; near 0 means on a boundary; negative means it probably belongs elsewhere.

Check	Meaning	Caveat
Silhouette	separation vs cohesion	**rewards spherical clusters** — flatters k-means, penalises DBSCAN
Davies–Bouldin	spread over separation; lower better	same geometric bias
Stability	re-run across seeds and bootstrap samples; do assignments agree?	the most honest of the three
Downstream use	does it improve a task you care about?	the only one that pays

Always report stability across at least five seeds. A clustering that changes when the seed does is a description of your random number generator.

Part II gate

Check	Threshold
Leakage demonstration reproduces	leaked R^2 exceeds honest by > 0.1 on pure noise
Pipeline vs transform-first inflation	measured and recorded
Your ridge against scikit-learn	coefficients agree to 1e-6
Soft-threshold formula vs a lasso solver in 1-D	exact zeros below α
Your worked Gini split numbers (0.18 vs 0.056)	exact
Worked confusion matrix (P 0.167, R 0.80, F1 0.276)	exact
Nested vs flat CV	nested ≤ flat; optimism gap reported
Every headline number	carries a bootstrap interval
Clustering report	includes stability over ≥ 5 seeds

Part III — Neural Networks in PyTorch

Everything in this part is Part I applied at scale. The tensors are Chapter 1's arrays with a gradient tape attached; autograd is Chapter 7's chain rule run in reverse; the optimiser is Chapter 6's update rule; the loss is Chapter 8's negative log-likelihood. If any of those four feel shaky, go back — this part will be mechanical if they are solid and mystifying if they are not.

16 · Tensors and automatic differentiation

16.1 A tensor is an array that remembers

A PyTorch tensor is a typed multi-dimensional array on some device *[PTG p.42–43]*. Set `requires_grad=True` and it gains something NumPy does not have: every operation records itself onto a graph, so the chain rule can later be replayed backwards *[PTG p.45]*.

```
import torch

x = torch.tensor([2.0], requires_grad=True)
y = x ** 3 + 2 * x                    # y = x³ + 2x,  dy/dx = 3x² + 2
y.backward()
print(x.grad)                         # 14.0 at x = 2
```

Three rules govern the graph, and almost every beginner bug is one of them:

1. **The graph is built during the forward pass and consumed by `backward()`.** Calling `backward()` twice needs `retain_graph=True`.
2. **Gradients accumulate.** `.grad` is added to, not overwritten. This is what makes gradient accumulation over micro-batches possible, and it is why `optimizer.zero_grad()` is mandatory — forget it and you are descending on the sum of every batch you have ever seen.
3. **Only leaves keep gradients.** Intermediate results drop theirs unless you ask with `retain_grad()`.

Detach from the graph when you do not want a path: `.detach()`, or a `with torch.no_grad():` block for inference and evaluation. Failing to do so in an eval loop is the most common cause of a memory leak that looks like a hardware problem — the graph for every evaluated batch is retained, because something still references it.

16.2 What `backward()` actually computes

A subtlety worth understanding, because it explains an error you will hit. Autograd does not build the full Jacobian; it computes a **vector–Jacobian product**, $v^{\mathsf{T}}J$. For a scalar loss, $v = 1$ and the product is the gradient, which is why `loss.backward()` works with no argument. Call `backward()` on a non-scalar and PyTorch raises, because it does not know which v you meant — `y.backward(torch.ones_like(y))` supplies it explicitly.

This is the cost argument of Chapter 7.2 realised in software: by only ever propagating row vectors leftwards through the chain, reverse mode never materialises an (output × parameter) Jacobian.

16.3 Prove autograd to yourself once

```
"""Check PyTorch's gradients against finite differences."""

import torch


def check_gradient(f, x: torch.Tensor, h: float = 1e-4) -> float:
    """Returns the largest gap between autograd and central differences.

    Args:
        f: Scalar-valued function of a tensor.
        x: Point of evaluation, requires_grad not needed.
        h: Step size for the numerical derivative.

    Returns:
        The maximum absolute difference across coordinates.
    """
    x = x.detach().double().requires_grad_(True)
    f(x).backward()
    analytic = x.grad.clone()
    numeric = torch.zeros_like(x)
    with torch.no_grad():
        for i in range(x.numel()):
            step = torch.zeros_like(x)
            step.view(-1)[i] = h
            numeric.view(-1)[i] = (f(x + step) - f(x - step)) / (2 * h)
    return float((analytic - numeric).abs().max())


fn = lambda t: (torch.sin(t) * torch.exp(-t ** 2)).sum()
print('max gap:', check_gradient(fn, torch.randn(5)))   # ~1e-10
```

Chapter 6's error analysis applies here too: use h around 1e-4 to 1e-6 and **double** precision. Gradient checking in float32 produces false alarms, because $\varepsilon \approx 10^{-7}$ puts the round-off floor above the discrepancy you are hunting.

17 · Layers, activations, and losses

17.1 Activations and their derivatives

The derivative matters as much as the function, because Chapter 7 showed it multiplies into every gradient flowing backwards through that unit *[PTG p.35–37]*.

Activation	f(z)	f′(z)	Notes
ReLU	max(0, z)	1 if z > 0 else 0	Default. No saturation for z > 0, so gradients survive depth. Dead units if the input is always negative.
Leaky ReLU	max(αz, z)	1 or α	Fixes dead units at the cost of one parameter.
Sigmoid	$1/(1+e^{-z})$	f(z)(1−f(z))	**Peaks at 0.25** (at z = 0). Output layers only.
Tanh	$(e^{z}-e^{-z})/(e^{z}+e^{-z})$	1 − f(z)²	Zero-centred, peaks at 1. Still saturates.
GELU	z·Φ(z)	smooth	The transformer default.

Two of those derivatives are worth deriving, as both reappear constantly.

Sigmoid. σ(z) = (1 + e^{−z})^{−1}, so

$$\sigma'(z) = e^{-z}/(1+e^{-z})^2 = \sigma(z)\cdot(1-\sigma(z))$$

Its maximum is at z = 0 where σ = ½, giving σ′ = **0.25** — the vanishing gradient coefficient from Chapter 7.

Tanh. tanh = (e^z − e^{−z})/(e^z + e^{−z}); differentiating the quotient gives 1 − tanh². Maximum 1 at z = 0, four times better than sigmoid, which is why tanh was preferred in RNNs long before ReLU existed.

17.2 Softmax and its Jacobian

Softmax turns a vector of scores into a distribution *[PTG p.37]*:

$$\text{softmax}(\mathbf{z})_i = e^{z_i} \,/\, \Sigma_j\, e^{z_j}$$

Never implement it naively. e^z overflows for z ≳ 710 in float64 and ≳ 88 in float32. Subtract the maximum first — the result is mathematically identical because the constant cancels top and bottom:

```python
def softmax(z: np.ndarray) -> np.ndarray:
    """Numerically stable softmax over the last axis."""
    shifted = z - z.max(axis=-1, keepdims=True)
    exp = np.exp(shifted)
    return exp / exp.sum(axis=-1, keepdims=True)
```

Its Jacobian has a tidy closed form. Writing p_i for the i-th output:

$$\partial p_i / \partial z_j = p_i(\delta_{ij} - p_j)$$

Derivation. For i = j, differentiate the quotient: the numerator contributes e^{z_i}·S and the denominator −e^{z_i}·e^{z_i}, over S², giving $p_i - p_i^2 = p_i(1 - p_i)$. For i ≠ j only the denominator depends on z_j, giving $-e^{z_i}e^{z_j}/S^2 = -p_i p_j$. ■

17.3 Losses are likelihoods

Chapter 8 did the real work; this is the dictionary *[PTG p.38–40]*:

Task	Loss	Assumption it encodes
Regression	MSE = (1/n)Σ(y − ŷ)²	Gaussian noise, constant variance
Regression, outliers	MAE / Huber	Laplace / mixed
Binary	BCE = −Σ[y log p + (1−y) log(1−p)]	Bernoulli
Multi-class	Cross-entropy = −Σ y log p	Categorical

The softmax + cross-entropy gradient, in full

This is the most important derivative in supervised deep learning, and the cancellation is beautiful. With one-hot target **y** and $L = -\Sigma_k y_k \log p_k$:

$$\partial L / \partial z_j = -\Sigma_k (y_k / p_k) \cdot \partial p_k / \partial z_j = -\Sigma_k (y_k / p_k) \cdot p_k(\delta_{kj} - p_j)$$

The p_k cancels:

$$= -\Sigma_k y_k(\delta_{kj} - p_j) = -y_j + p_j \Sigma_k y_k = p_j - y_j$$

using $\Sigma_k y_k = 1$ for a one-hot target. So

$$\partial L / \partial z = p - y$$

■ — the same *prediction minus truth* as linear regression (Chapter 4) and logistic regression (Chapter 12).

Two practical consequences follow directly from that derivation:

- **Use the fused loss.** `CrossEntropyLoss` and `BCEWithLogitsLoss` take *logits* and compute this directly via log-sum-exp, never forming p and then logging it. Applying softmax yourself and then taking a log is the classic route to NaN, because log(0) happens the moment a probability underflows.
- **The gradient never saturates.** Unlike squared error on a sigmoid output — where an σ′ factor of 0.25 or less survives — here the activation derivative cancelled entirely. A confidently wrong prediction produces a gradient of magnitude ≈ 1, so the network can recover from it.

17.4 Initialisation, derived

Initialise all weights to zero and every unit in a layer computes the same thing and receives the same gradient forever — the network has one unit pretending to be many. Initialise too large and activations saturate or explode. The right scale follows from a variance calculation.

For $z = \Sigma_i w_i x_i$ with independent, zero-mean w and x:

$$\mathrm{Var}(z) = n_{\text{in}} \cdot \mathrm{Var}(w) \cdot \mathrm{Var}(x)$$

To keep Var(z) = Var(x) through the layer we need Var(w) = 1/n_in. **Xavier / Glorot** compromises between the forward and backward passes with Var(w) = 2/(n_in + n_out), appropriate for tanh and sigmoid.

ReLU zeroes half its inputs, halving the variance, so the factor must be doubled: **He / Kaiming**, Var(w) = 2/n_in. That factor of two is the entire difference between the two schemes, and it is the difference between a 50-layer ReLU network training and dying.

PyTorch's defaults are sensible; override deliberately or not at all.

17.5 Normalisation

Both common normalisers subtract a mean and divide by a standard deviation, then re-scale with learned parameters γ and β:

$$\hat{x} = (x - \mu)/\sqrt{(\sigma^2 + \varepsilon)}, \quad y = \gamma\hat{x} + \beta$$

The difference is **which axis** the statistics come from:

	Statistics over	Depends on batch?	Train ≠ eval?	Used in
BatchNorm	the batch, per channel	yes	yes — running averages at eval	CNNs
LayerNorm	the features, per sample	no	no	transformers, RNNs

BatchNorm's batch dependence is why small batches degrade it, and why it needs separate train/eval behaviour — a frequent source of "my model scores differently in eval mode" confusion. LayerNorm has neither problem, which is why sequence models standardised on it.

---

18 · The training loop

18.1 Dataset and DataLoader

A `Dataset` answers two questions — how many items, and what is item i *[PTG p.76]*. A `DataLoader` batches, shuffles, and parallelises the fetching.

```
"""A minimal Dataset and the training loop with no framework helpers."""

import numpy as np
import torch
from torch.utils.data import DataLoader, Dataset


class ArrayDataset(Dataset):
    """Wraps NumPy arrays as a PyTorch dataset.

    Attributes:
        x: Feature tensor of shape (n, d).
        y: Target tensor of shape (n, 1).
    """

    def __init__(self, x: np.ndarray, y: np.ndarray) -> None:
        """Stores the arrays as float32 tensors.

        Args:
            x: Features of shape (n, d).
            y: Targets of shape (n,).

        Raises:
            ValueError: If the lengths differ.
        """
        if len(x) != len(y):
            raise ValueError(f'length mismatch: {len(x)} vs {len(y)}')
        self.x = torch.tensor(x, dtype=torch.float32)
        self.y = torch.tensor(y, dtype=torch.float32).reshape(-1, 1)

    def __len__(self) -> int:
        """Returns the number of samples."""
        return len(self.x)

    def __getitem__(self, index: int) -> tuple:
        """Returns the (features, target) pair at `index`."""
        return self.x[index], self.y[index]


def train(model, loader, epochs: int, lr: float) -> list:
    """Trains a model and returns the per-epoch mean loss.

    Args:
        model: A torch.nn.Module.
        loader: A DataLoader over the training set.
        epochs: Number of passes over the data.
        lr: Learning rate.

    Returns:
        A list of mean losses, one per epoch.
    """
    optimiser = torch.optim.SGD(model.parameters(), lr=lr)
    loss_fn = torch.nn.MSELoss()
    history = []
    model.train()
    for _ in range(epochs):
        running, batches = 0.0, 0
        for xb, yb in loader:
            optimiser.zero_grad()        # 1. clear accumulated gradients
            prediction = model(xb)       # 2. forward
            loss = loss_fn(prediction, yb)
            loss.backward()              # 3. backward
            optimiser.step()             # 4. update
            running += float(loss)
            batches += 1
```

```
        history.append(running / batches)
    return history
```

Those four lines in the inner loop are the entire algorithm. Everything PyTorch Lightning, Hugging Face Trainer and the rest provide is scaffolding around them *[PTG p.351]* — and you should be able to write them from memory before you let a framework write them for you.

18.2 Batch size

Batching trades gradient quality against throughput *[PTG p.72]*. The mini-batch gradient is an unbiased estimator of the full gradient, and by Chapter 8's variance result its standard error scales as

$$\mathrm{SE} \propto \sigma/\sqrt{B}$$

so quadrupling the batch halves the gradient noise — the same $\sqrt{n}$ law, charging its usual rate.

Batch	Gradient	Hardware	Generalisation
1 (SGD)	very noisy	idle GPU	noise acts as regularisation
32–256	good compromise	efficient	the usual default
Very large	near-exact	saturated	often worse; needs LR scaling and warm-up

Because noise scales as $1/\sqrt{B}$ while the step size scales linearly, keeping the *ratio* of noise to step constant when you grow the batch requires raising the learning rate — the origin of the **linear scaling rule** (multiply the LR by the same factor as the batch, with a warm-up to survive the early steps).

The noise in small-batch SGD is not purely a cost: it helps escape sharp minima, which is why simply raising the batch size for speed, without touching the learning rate, usually hurts.

18.3 Optimisers

Optimiser	Update	When
SGD	$w \leftarrow w - h\cdot\nabla L$	with momentum, still the strongest for vision
SGD + momentum	$v \leftarrow \beta v - h\nabla L$; $w \leftarrow w + v$	Chapter 6's heavy ball
Adam	per-parameter adaptive steps	the safe default elsewhere
AdamW	Adam with decoupled weight decay	the right Adam for regularised training

Adam, line by line

$$m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t \quad \cdot \quad v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^{\,2}$$

$$\hat{m}_t = m_t/(1-\beta_1{}^t) \quad \cdot \quad \hat{v}_t = v_t/(1-\beta_2{}^t)$$

$$w_t = w_{t-1} - h \cdot \hat{m}_t/(\sqrt{\hat{v}_t} + \varepsilon)$$

m is a running mean of the gradient (momentum); v is a running mean of its square (a per-parameter scale). Dividing by √v gives each parameter its own effective step — which is why Adam is forgiving of the bad conditioning that cripples plain SGD (Chapter 6).

Why the bias correction? $m_0 = 0$, so after one step $m_1 = (1-\beta_1)g_1$, which underestimates the gradient by a factor of $(1-\beta_1) = 0.1$ at the usual $\beta_1 = 0.9$. Taking expectations of the recursion gives $\mathbb{E}[m_t] \approx (1-\beta_1{}^t)\mathbb{E}[g]$, so dividing by $(1-\beta_1{}^t)$ removes the bias exactly. The correction matters most in the first few dozen steps and decays to nothing — but without it, early updates are badly scaled, which in a short fine-tune is the whole run.

In plain Adam, L2 weight decay enters through the gradient and is then divided by √v, so strongly-updated parameters get *less* decay — the opposite of the intent. **AdamW** applies the decay directly to the weights instead, which is why it is standard for transformers.

Schedules matter more than people expect. Cosine decay with a short warm-up is a reliable default. To choose the peak rate, run an **LR range test** *[TSP ch.6 p.245]*: increase the rate geometrically over one epoch, plot loss against rate, and take roughly an order of magnitude below the minimum. You have seen that curve already in Chapter 6 — the divergence edge is the 2/L edge.

18.4 Determinism and checkpoints

"It's random" is not an acceptable explanation for a run you cannot reproduce.

```python
def set_seed(seed: int) -> None:
    """Seeds every generator that affects a training run."""
    import random
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    torch.cuda.manual_seed_all(seed)
    torch.use_deterministic_algorithms(True, warn_only=True)
    torch.backends.cudnn.benchmark = False
```

Save the *state dict*, not the pickled model *[PTG p.80]*, and save enough to resume rather than merely to infer: model state, optimiser state (Adam's m and v are parameters too), scheduler state, epoch, and the seed. A checkpoint that cannot resume training is half a checkpoint.

18.5 Reading a loss curve

Symptom	Likely cause
Loss flat from step 0	LR far too low, or gradients not reaching (missing `zero_grad`, detached graph)
Loss → NaN	LR above the stability limit, or log(0) from an unfused softmax
Loss explodes then recovers	one bad batch; clip gradients
Train falls, validation rises	overfitting — regularise, augment, or stop earlier
Both fall then plateau high	underfitting — more capacity, longer schedule, better features
Validation below training	not a miracle: dropout is active in training, or your split leaks

19 · Convolution

19.1 Why not a dense layer

A 224×224 RGB image flattened is 150,528 inputs; one dense layer of 1,000 units is 150 million parameters, and it learns nothing about the fact that neighbouring pixels are related. Convolution encodes two priors *[PTG p.128–129]*:

- **Locality** — features depend on nearby pixels;
- **Translation equivariance** — an edge is an edge wherever it appears, so the same kernel applies everywhere.

Weight sharing turns those priors into a parameter count that does not scale with image size.

19.2 The arithmetic you must know

$$\text{out} = \lfloor (\text{in} + 2\cdot\text{pad} - \text{dilation}\cdot(\text{kernel} - 1) - 1) \,/\, \text{stride} \rfloor + 1$$

Worked example

A 32×32 input, 3×3 kernel, stride 1, padding 1:

$$(32 + 2 - 1\cdot(3-1) - 1)/1 + 1 = (32 + 2 - 2 - 1) + 1 = \mathbf{32}$$

Size preserved — which is why odd kernels with pad = (k−1)/2 dominate. Same input with stride 2:

$$\lfloor (32 + 2 - 2 - 1)/2 \rfloor + 1 = \lfloor 15.5 \rfloor + 1 = \mathbf{16}$$

Halved, which is how you downsample without pooling.

Parameters for C_in → C_out with k×k kernels: C_out·(C_in·k² + 1), **independent of the input resolution**. A 3×3 layer from 64 to 128 channels is 128·(64·9 + 1) = 73,856 parameters whether the image is 32×32 or 1024×1024.

The **receptive field** grows linearly with depth for stride-1 layers: two stacked 3×3 convolutions see 5×5, three see 7×7 — with fewer parameters (2·9 = 18 vs 49 per channel pair) and more nonlinearity than a single 7×7. That observation is most of what VGG contributed, and it is implemented in `papers/09_resnet` in this repository. Dilation grows the field *exponentially* without pooling — `papers/11_dilated_convolutions`.

19.3 Convolution is a matrix multiply

Under the hood, `im2col` unfolds every patch into a row, turning convolution into the GEMM that Chapter 3 described and that GPUs are built for:

```
def conv2d_via_im2col(x: np.ndarray, kernel: np.ndarray) -> np.ndarray:
    """Single-channel 2-D convolution expressed as a matrix product.

    Args:
        x: Input of shape (H, W).
        kernel: Kernel of shape (kh, kw).

    Returns:
        The valid cross-correlation, shape (H-kh+1, W-kw+1).
    """
    kh, kw = kernel.shape
    out_h, out_w = x.shape[0] - kh + 1, x.shape[1] - kw + 1
    patches = np.lib.stride_tricks.sliding_window_view(x, (kh, kw))
    cols = patches.reshape(out_h * out_w, kh * kw)
    return (cols @ kernel.ravel()).reshape(out_h, out_w)
```

Note that this, like PyTorch's `conv2d`, computes *cross-correlation* — the kernel is not flipped. Since the kernel is learned, the distinction is irrelevant to the result and matters only when comparing against a signal-processing reference.

The backward pass

Once convolution is a matrix multiply, Chapter 7's rules apply unchanged. With Y = W·X_col:

$$\partial L/\partial W = \delta \cdot X_{col}^{T} \quad \cdot \quad \partial L/\partial X_{col} = W^{T} \cdot \delta$$

The second is then folded back into image shape (`col2im`), summing the contributions of overlapping patches — which is itself a convolution with the kernel flipped. **The gradient of a convolution is a convolution**, which is why conv layers are as cheap backwards as forwards.

---

20 · Autoencoders and the variational autoencoder

20.1 Autoencoders

An autoencoder learns to reconstruct its input through a narrow bottleneck *[PTG p.220]*. With linear layers and squared error it recovers the PCA subspace of Chapter 5 — the same optimum, by a slower route, because minimising reconstruction error over a k-dimensional linear subspace *is* derivation (b) of Section 5.5. (It finds the subspace, not the orthonormal basis: the solution is unique only up to an invertible k×k transform.)

The honest benchmark is therefore PCA at the same latent dimension. If your autoencoder cannot beat it, it has learned nothing a 1901 algorithm did not — and the fix is nonlinearity, not more epochs.

20.2 The VAE, derived

A variational autoencoder models the data as generated from a latent **z** *[PTG p.230]*. We want log p(x) and cannot compute it, because it requires integrating over every possible z. So bound it.

For *any* distribution q(z|x), start from the definition and insert q twice:

$$\log p(x) = \log \int p(x,z)\, dz = \log \int q(z|x) \cdot (p(x,z)/q(z|x))\, dz$$

Jensen's inequality (Chapter 8, log is concave) moves the log inside:

$$\log p(x) \geq \int q(z|x) \cdot \log(p(x,z)/q(z|x))\, dz = \mathbb{E}_q[\log p(x|z)] - D_{KL}(q(z|x) \,\|\, p(z))$$

That right-hand side is the **ELBO**. The gap between it and log p(x) is exactly D_KL(q(z|x) ‖ p(z|x)) — non-negative by Gibbs, zero only when q is the true posterior. So **maximising the ELBO does two things at once**: it pushes up a lower bound on the likelihood, and it pushes q towards the true posterior.

Two terms with plain meanings: **reconstruct the input well**, and **keep the latent distribution close to the prior**. For Gaussian q and standard normal prior the KL has a closed form:

$$D_{KL} = -\tfrac{1}{2} \Sigma_j \left(1 + \log \sigma_j^2 - \mu_j^2 - \sigma_j^2\right)$$

The reparameterisation trick

Sampling z ~ q blocks gradients: you cannot differentiate through a sampler whose parameters determine the distribution being sampled. Rewrite the sample as a deterministic function of the parameters and *external* noise:

$$z = \mu + \sigma \odot \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, I)$$

Now the randomness lives in ε, which carries no parameters, and $\partial z/\partial \mu = 1$, $\partial z/\partial \sigma = \varepsilon$ are both well-defined. The path from loss to encoder stays differentiable. (The alternative, score-function estimators, work without this trick but have variance orders of magnitude higher — which is why VAEs became practical only with the reparameterisation.)

```python
def vae_loss(x_hat, x, mu, logvar):
    """ELBO as a minimisable loss: reconstruction plus KL.

    Args:
        x_hat: Reconstruction, shape (batch, d).
        x: Original input, shape (batch, d).
        mu: Latent means, shape (batch, k).
        logvar: Latent log-variances, shape (batch, k).

    Returns:
        A tuple (total, reconstruction, kl), each a scalar tensor.
    """
    recon = torch.nn.functional.mse_loss(x_hat, x, reduction='sum')
    kl = -0.5 * torch.sum(1 + logvar - mu.pow(2) - logvar.exp())
    return recon + kl, recon, kl
```

The network predicts `logvar` rather than σ because log-variance is unconstrained — predicting σ directly would need a positivity constraint, and exp() enforces it for free while keeping gradients well-scaled.

Monitor both terms separately. If the KL collapses to zero the encoder has stopped using the latent space and you have an ordinary autoencoder wearing a VAE's clothes. Watching only the sum hides this completely.

---

21 · Attention and transformers

21.1 Scaled dot-product attention

Every position builds a query, and every position offers a key and a value. Relevance is a dot product — Chapter 2's inner product used as a similarity — and the output is a weighted average of values *[PTG p.320]*:

$$\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_k}) \cdot V$$

Why √d_k, derived

The scaling is not decoration. Suppose query and key entries are independent with zero mean and unit variance. Their dot product is a sum of d_k such products, so by Chapter 8's variance-of-a-sum:

$$\text{Var}(\mathbf{q}\cdot\mathbf{k}) = \Sigma_{i=1}^{d_k} \text{Var}(q_i k_i) = d_k$$

Standard deviation √d_k. At d_k = 64 that is ±8 before scaling, and a softmax over logits spread ±8 is essentially a hard maximum — one entry near 1, the rest near 0. From Section 17.2's Jacobian, $p_i(\delta_{ij} - p_j) \to 0$ as p → one-hot, so **the gradient vanishes**.

Dividing by √d_k restores unit variance, keeps the softmax in its responsive range, and keeps gradients alive. ■ One divided square root is the difference between a trainable mechanism and a dead one.

Multi-head attention runs h of these in parallel on projected subspaces and concatenates, letting different heads attend to different kinds of relation — and since each head uses d_k = d/h, the total cost is roughly that of single-head attention at full width.

Cost: QK^T is (n × n), so attention is **O(n²)** in sequence length, in time and memory. That single fact drives context-length limits, and every efficient-attention variant exists to attack it.

21.2 The rest of the block

Component	Purpose
Positional encoding	attention is permutation-invariant; without this, "dog bites man" equals "man bites dog"
Residual connection	a gradient highway: $\partial(x + f(x))/\partial x = 1 + f'$, so gradients survive depth — `papers/09_resnet`
Layer normalisation	stabilises scale per token; pre-norm placement trains more reliably at depth
Feed-forward (×4 width)	per-position processing; most of the parameters live here
Causal mask	in a decoder, prevents attending to the future — the architectural form of Chapter 10's lookforward leakage

The residual entry deserves emphasis. Chapter 7 showed gradients die through products of small factors; a residual branch contributes an **additive 1** to that product, so the gradient has a path that cannot be attenuated. It is the same trick as the LSTM's additive cell state (Chapter 24), arrived at independently.

Implement one yourself before using `nn.TransformerEncoder`. `papers/13_attention` and `papers/14_annotated_transformer` walk through it; the gate is that your attention output matches PyTorch's to 1e-5 on the same weights.

Part III gate

Check	Threshold
Autograd vs finite differences	< 1e-9 in double precision
Hand-written backprop vs autograd on a 2-layer net	< 1e-7
Your softmax Jacobian vs numerical	< 1e-6
Softmax + CE gradient equals p − y	< 1e-6
Naive vs stable softmax at z = 1000	naive overflows; stable does not
Conv output sizes vs the formula	exact at stride 1 and 2
Your `conv2d_via_im2col` vs `F.conv2d`	< 1e-5
Checkpoint reload reproduces logits	bit for bit
Autoencoder vs PCA at equal latent dim	compared, result reported either way
VAE run	reconstruction and KL logged separately
Your attention vs `nn.MultiheadAttention`	< 1e-5

Part IV — Sequences and Uncertainty

Temporal data punishes every evaluation sin immediately and without mercy. A shuffled split on a time series will hand you a beautiful score and a useless model, and nothing in the training loop will warn you. This part is therefore as much about *measurement* as about architecture — which is why it is the part that most separates a professional from a competent beginner.

22 · The structure of temporal data

22.1 What makes it different

Three properties break the i.i.d. assumption every method in Part II relies on *[TSP ch.2]*:

- **Order carries information.** Shuffling destroys it.
- **Observations are correlated with their own past** — autocorrelation.
- **The generating process drifts.** Tomorrow may not be drawn from today's distribution.

The last one is the deepest. It means a model is always an extrapolation, and that no amount of held-out data proves it will keep working.

22.2 Decomposition

A series is usefully read as level + trend + seasonality + remainder *[TSP p.65]*. Two combination rules:

$$\text{additive: } y_t = T_t + S_t + R_t \quad \cdot \quad \text{multiplicative: } y_t = T_t \times S_t \times R_t$$

Use additive when the seasonal swing is a constant *size [TSP p.66]*, multiplicative when it is a constant *proportion [TSP p.70]*.

Worked example — which one, and the log that converts them

Two series, each with a December peak:

Year	Series A (Dec − baseline)	Series B (Dec ÷ baseline)
1	100 − 1000 = +100	1100 ÷ 1000 = 1.10
2	1200 − 1100 = +100	1210 ÷ 1100 = 1.10
3	1300 − 1200 = +100	1331 ÷ 1210 = 1.10

A is additive: the December bump is +100 regardless of level. B is multiplicative: the bump is +10% and therefore grows in absolute terms. Take logs of B:

$$\log(T \cdot S \cdot R) = \log T + \log S + \log R$$

and it becomes additive. **That is the entire reason logging is the first thing done to a business series** — it converts proportional seasonality into the constant kind, which every additive method can then handle.

STL *[TSP p.73]* does the decomposition robustly with LOESS, handles a changing seasonal shape, and resists outliers. It is the sensible default.

22.3 ACF and PACF

The **autocorrelation function** is the correlation of the series with itself at lag k *[TSP p.84]*:

$$\rho(k) = \Sigma_{t=k+1}^{n}(y_t - \bar{y})(y_{t-k} - \bar{y}) \,/\, \Sigma_{t=1}^{n}(y_t - \bar{y})^2$$

Derivation — the ACF of an AR(1)

Knowing one ACF analytically makes every empirical plot readable. Let $y_t = \varphi y_{t-1} + \varepsilon_t$ with $|\varphi| < 1$ and white noise ε. The stationary variance satisfies

$$\gamma_0 = \mathrm{Var}(y) = \varphi^2\gamma_0 + \sigma^2 \implies \gamma_0 = \sigma^2/(1-\varphi^2)$$

For the covariance at lag k, multiply the recursion by y_{t-k} and take expectations. Since ε_t is independent of the past, that term drops:

$$\gamma_k = \varphi\gamma_{k-1} \implies \gamma_k = \varphi^k\gamma_0 \implies \rho(k) = \varphi^k$$

■ **The ACF of an AR(1) decays geometrically.** With $\varphi = 0.7$: $\rho(1) = 0.70$, $\rho(2) = 0.49$, $\rho(3) = 0.34$ — exactly the decaying tail you see in real plots, and the reason a slow decay suggests φ near 1, i.e. near non-stationarity.

The **partial** autocorrelation *[TSP p.86]* gives the correlation at lag k with the intervening lags' effects removed. For that same AR(1), PACF(1) = φ and PACF(k) = 0 for $k > 1$ — all of the lag-2 correlation was *inherited* through lag 1, none of it direct. That contrast is what the two plots are for:

Pattern	Reading
ACF decays slowly, almost linearly	non-stationary; difference it
ACF decays geometrically, PACF cuts off after p	autoregressive, order p
ACF cuts off after q, PACF decays	moving average, order q
ACF spikes at 7, 14, 21	weekly seasonality

For a neural model you are not fitting ARIMA, but these plots still tell you *which lags to feed it* — the cheapest feature engineering in forecasting.

```python
def acf(series: np.ndarray, max_lag: int) -> np.ndarray:
    """Sample autocorrelation up to max_lag.

    Args:
        series: One-dimensional series.
        max_lag: Highest lag to compute.

    Returns:
        Array of length max_lag + 1, starting at lag 0 (always 1.0).
    """
    y = np.asarray(series, dtype=np.float64)
    y = y - y.mean()
    denom = float(y @ y)
    return np.array([float(y[k:] @ y[:len(y) - k]) / denom
                     for k in range(max_lag + 1)])
```

22.4 Stationarity

Weak stationarity means the mean is constant, the variance is finite and constant, and the covariance γ(k) depends only on the lag k, never on where in the series you are *[TSP p.92]*. Most statistical methods assume it; most real series lack it.

Three tests, and they do not agree by construction *[TSP p.97]*:

Test	Null hypothesis	Rejecting means
ADF	has a unit root (non-stationary)	stationary
KPSS	is stationary	non-stationary
PP	has a unit root, robust to heteroskedasticity	stationary

ADF and KPSS have *opposite* nulls, so all four combinations occur:

ADF	KPSS	Reading
rejects	fails to reject	stationary — both agree
fails to reject	rejects	non-stationary — both agree
fails to reject	fails to reject	**not enough data to decide**
rejects	rejects	likely trend-stationary, or heteroskedastic

Record both outcomes. Choosing whichever test agrees with your plan is the most common quiet dishonesty in applied forecasting, and it is invisible in the final report unless you make it visible.

What differencing does

Fix non-stationarity by differencing, $\Delta y_t = y_t - y_{t-1}$. On a random walk ($\varphi = 1$, the unit-root case) differencing returns exactly the white-noise innovations — which is why a random walk's best forecast is its last value, the naive baseline of Section 23.3.

Beware **over-differencing**: applying Δ to series that is already stationary introduces a negative lag-1 autocorrelation of -0.5 that was not there before, and your diagnostics will now show structure you created. Difference the minimum number of times the tests require, and no more.

22.5 Structure determines the splitter

[TSP p.103–106] distinguishes four shapes, and getting this wrong guarantees the wrong evaluation:

Structure	Example	Splitter
Univariate	one store's daily sales	fixed origin / rolling
Multivariate	several series predicted jointly	as above, all aligned
Multivariable	one target, exogenous drivers	as above, drivers lagged
Panel	10,000 stores × 3 years	**time-aware *and* group-aware**

23 · Honest temporal evaluation

23.1 Never shuffle

A random split lets the model interpolate between points it has already seen, which is not the task. Production requires extrapolation past the end of the data. Three valid designs:

Fixed origin *[TSP p.126]* — train on everything up to T, test after. Simple, one estimate, high variance.

Expanding window *[TSP p.132]* — the origin moves forward and training history grows. Uses all data, mirrors a model retrained periodically.

Rolling window *[TSP p.134]* — fixed-length history slides forward. Constant training size across folds, so folds are comparable, and it adapts if the process drifts.

Worked fold boundaries

n = 120, four folds, horizon 10, window 50:

Fold	Train	Test
0	[0, 50)	[50, 60)
1	[10, 60)	[60, 70)
2	[20, 70)	[70, 80)
3	[30, 80)	[80, 90)

Every training window is exactly 50 long, every test block strictly follows its training block, and the last index used is 90 — so you need n ≥ window + folds × horizon = 90, and the remaining 30 points are available for a final held-out test. Compare with the expanding design, where fold 3 would train on [0, 80): more data, but fold 0 and fold 3 are no longer comparable because they saw different amounts of history.

```python
def rolling_window_cv(n: int, n_folds: int, horizon: int, window: int) -> list:
    """Builds rolling-origin folds whose training length is constant.

    Args:
        n: Length of the series.
        n_folds: Number of folds.
        horizon: Test length per fold.
        window: Fixed training-window length.

    Returns:
        A list of (train_idx, test_idx, description) tuples.

    Raises:
        ValueError: If the series is too short for the design.
    """
    needed = window + n_folds * horizon
    if needed > n:
        raise ValueError(f'need {needed} points, have {n}')
    folds = []
    for i in range(n_folds):
        end = window + i * horizon
        folds.append((np.arange(end - window, end),
                      np.arange(end, end + horizon),
                      f'train [{end - window},{end}) < test [{end},{end + horizon})'))
    return folds


def assert_no_lookforward(folds: list) -> None:
    """Fails if any training index reaches past its test window.

    Raises:
        AssertionError: If a fold leaks the future into training.
    """
    for train_idx, test_idx, label in folds:
        if train_idx.max() >= test_idx.min():
            raise AssertionError(f'lookforward leak in {label}')
```

Print the boundaries into your report and assert them in code. A reviewer who cannot see them cannot check the evaluation, and an assertion that runs is worth more than a convention you intend to follow.

23.2 Metrics, and the two traps

Start with the ordinary ones *[TSP p.138–144]*: MAE, MSE, RMSE as in Chapter 14. Then the two failures specific to forecasting.

Trap 1 — percentage errors break at zero. APE = $|y - \hat{y}|/|y|$ is undefined when $y = 0$ *[TSP p.144]*. Intermittent demand is full of zeros. An implementation that returns infinity poisons every aggregate downstream in silence; the right behaviour is to refuse.

Trap 2 — MAPE is not scale invariant. *[TSP p.143]* makes this vivid with a temperature table. Reproduce it: a forecast errs by 1° on a true value of 20 °C.

$$\text{MAPE in Celsius} = 1/20 = 5.0\% \quad \cdot \quad \text{MAPE in Kelvin} = 1/293.15 = 0.34\%$$

Same model, same error, same physical reality — and a fifteenfold difference in the reported metric, produced entirely by the choice of zero point. Any metric that behaves this way cannot compare series.

The fix is to scale by a **naive forecast computed on the training data** *[TSP p.150]*:

$$\text{MASE} = \text{MAE(model)} \,/\, \text{MAE(naive on train)} \quad \cdot \quad \text{RMSSE} = \text{RMSE(model)} \,/\, \text{RMSE(naive on train)}$$

Worked MASE

Training series: 10, 12, 11, 13, 12, 14. The one-step naive differences are |12−10|, |11−12|, |13−11|, |12−13|, |14−12| = 2, 1, 2, 1, 2, so

$$\text{naive scale} = (2+1+2+1+2)/5 = 1.6$$

Now test: truth 15, 14, 16; forecast 14, 15, 15. Absolute errors 1, 1, 1, so MAE = 1.0 and

$$\text{MASE} = 1.0 \,/\, 1.6 = 0.625$$

Below 1, so the model beats the naive forecast — by 37.5%. Add 273.15 to every number and recompute: the differences are unchanged, the errors are unchanged, and MASE is **still 0.625**. That invariance is why it is this text's primary forecasting metric.

> **The denominator uses training data only.** Computing the naive baseline on the test window is a leak, and an easy one to commit because the code looks symmetric.

```python
def naive_scale(y_train: np.ndarray, seasonality: int = 1) -> float:
    """MAE of the seasonal naive forecast on the TRAINING series.

    Args:
        y_train: Training portion, in time order.
        seasonality: Seasonal period; 1 is the random walk.

    Returns:
        The scaling constant for MASE.

    Raises:
        ValueError: If the series is too short or constant.
    """
    y = np.asarray(y_train, dtype=np.float64)
    if y.size <= seasonality:
        raise ValueError('training series too short for this seasonality')
    scale = float(np.mean(np.abs(y[seasonality:] - y[:-seasonality])))
    if scale == 0.0:
        raise ValueError('constant training series; MASE undefined')
    return scale


def mase(y_true, y_pred, y_train, seasonality: int = 1) -> float:
    """Mean absolute scaled error."""
    mae = float(np.mean(np.abs(np.asarray(y_true) - np.asarray(y_pred))))
    return mae / naive_scale(y_train, seasonality)
```

23.3 Baselines you must beat

Baseline	Prediction	Optimal when
Naive	$\hat{y}_{t+h} = y_t$	the series is a random walk
Seasonal naive	$\hat{y}_{t+h} = y_{t+h-s}$	strong stable seasonality
Drift	last value + average slope	random walk with drift
Mean	the training mean	the series is stationary noise

The first row is not a joke. Section 22.4 showed that differencing a random walk leaves white noise — meaning *nothing* is predictable beyond the last value, and the naive forecast is provably optimal. On short or noisy series these baselines frequently win.

Report it when they do. A neural network losing to seasonal naive is a finding about the problem — the signal is weak, or the horizon too long — and suppressing it is the single most common dishonesty in applied forecasting.

23.4 Horizon

Error grows with horizon, and a single averaged number hides the shape *[TSP p.156]*. For an AR(1), the h-step forecast is $\varphi^h y_t$ and its error variance is

$$\sigma^2 \cdot (1 + \varphi^2 + \varphi^4 + \cdots + \varphi^{2(h-1)}) = \sigma^2 \cdot (1 - \varphi^{2h})/(1 - \varphi^2)$$

which rises to the unconditional variance $\sigma^2/(1-\varphi^2)$ as h grows. **Beyond some horizon, your model knows nothing that the series' long-run mean does not** — and that horizon is a property of the data, not of your architecture. Plot metric against h; a model excellent at h = 1 and useless at h = 7 is a different product from one that degrades gently.

24 · Neural forecasting

24.1 Windowing

Supervised learning needs (X, y) pairs, so slide a window: inputs y_{t−L+1..t}, target y_{t+1..t+h}. Two rules:

1. **Build windows inside the fold**, after splitting. A window that straddles the boundary puts future values into a training input *[TSP p.161]*.
2. **Scale with training statistics only**, and invert before computing metrics.

Rule 1 has a quiet corollary: with a window length L, the first L−1 points of each test block cannot be predicted without reaching back into training data. That is legitimate (the past is known at prediction time) but it means your effective test size is smaller than it looks, and the folds overlap in inputs even when they do not overlap in targets.

24.2 Recurrent networks

An RNN carries a hidden state forward *[TSP ch.8 p.351]*:

$$h_t = \tanh(W_h h_{t-1} + W_x x_t + b)$$

Why gradients vanish or explode, exactly

Backpropagation through time unrolls this into a network whose depth is the sequence length. The gradient of a loss at time T with respect to a state at time t involves the product

$$\partial h_T / \partial h_t = \Pi_{k=t+1}^{T} \partial h_k / \partial h_{k-1} = \Pi_k \operatorname{diag}(1 - h_k^2) \cdot W_h$$

Bound the norm: ‖∂h_T/∂h_t‖ ≤ (‖W_h‖·max|tanh′|)^{T−t}. Since |tanh′| ≤ 1, if the largest singular value of W_h is below 1 the product **decays geometrically** in T − t; if it is above 1 it **explodes** geometrically. ■

Long dependencies are therefore unlearnable in a plain RNN except in the knife-edge case where the spectral radius sits at exactly 1.

LSTM solves it with gates and an additive cell state:

$$f_t = \sigma(W_f[h_{t-1}, x_t]) \;\cdot\; i_t = \sigma(W_i[h_{t-1}, x_t]) \;\cdot\; o_t = \sigma(W_o[h_{t-1}, x_t])$$

$$c_t = f_t \odot c_{t-1} + i_t \odot \tanh(W_c[h_{t-1}, x_t]) \;\cdot\; h_t = o_t \odot \tanh(c_t)$$

The decisive line is the cell update. Its derivative along the cell path is

$$\partial c_t / \partial c_{t-1} = f_t$$

a *gate value*, not a weight matrix times an activation derivative. With the forget gate near 1, the gradient passes through essentially unattenuated for hundreds of steps — the **constant error carousel**. It is the same additive trick as the residual connection of Chapter 21, discovered two decades earlier.

`papers/02_Understanding_LSTM` implements all four gates from scratch; the GRU merges them into two and is usually as good with fewer parameters.

Always **clip gradients** when training recurrent models — `clip_grad_norm_` at 1.0 is standard and prevents a single exploding batch from destroying the run. Clipping addresses the explosion half of the analysis above; the gates address the vanishing half.

24.3 Transformers for forecasting

Attention (Chapter 21) removes the sequential bottleneck: every position sees every other in one step, so a long dependency is one hop away rather than L. The trade is explicit:

	RNN / LSTM	Transformer
Path length between distant positions	O(L)	**O(1)**
Compute per layer	$O(L \cdot d^2)$	$O(L^2 \cdot d)$
Parallel over time?	no	**yes**
Inductive bias for order	built in	**must be supplied**

That last row is why positional encoding is mandatory, and why transformers need more data than an LSTM to reach the same place on small series — they must *learn* what recurrence assumes.

Use a **causal mask** so position t attends only to ≤ t. Omitting it is lookforward leakage implemented in matrix form, and it produces spectacular validation numbers that collapse in production.

Benchmark honestly: same folds, same scaler, same metric, matched parameter and compute budgets. Transformers do not automatically beat an LSTM on a few thousand points, and published comparisons that say otherwise usually differ in budget.

25 · Uncertainty: conformal prediction

25.1 Why a point forecast is not enough

"Demand will be 1,000" supports no decision. "Demand will be between 900 and 1,100 with 90% probability" supports inventory, staffing and risk. Most intervals in practice come from distributional assumptions that are wrong — Gaussian residuals on heavy-tailed data — and are therefore too narrow exactly when they matter *[TSP p.311]*.

Conformal prediction gives intervals with a finite-sample coverage guarantee and almost no assumptions: only **exchangeability**, meaning the joint distribution is invariant to reordering. (Weaker than i.i.d., and notably *not* satisfied by drifting time series — see 25.4.)

25.2 The mechanism

The idea is a ranking, not a distribution *[TSP p.316–317]*. Define a **non-conformity score** measuring how badly a point fits the model — absolute error is the usual choice. Score every point in a held-out calibration set. A new prediction's plausible range is then bounded by a quantile of those scores.

Split conformal, in full:

1. Split into train / calibration / test.
2. Fit the model on train only.
3. Score residuals $s_i = |y_i - \hat{y}_i|$ on calibration.
4. Take q = the $\lceil (n+1)(1-\alpha) \rceil$-th smallest of those scores.
5. Predict $\hat{y} \pm q$.

Why coverage is guaranteed — the rank argument

This is the whole theoretical content, and it is one paragraph.

Under exchangeability, the test score s_{n+1} and the calibration scores $s_1 \ldots s_n$ are exchangeable, so **s_{n+1} is equally likely to occupy any of the n+1 possible ranks** in the combined set. The interval fails to cover exactly when s_{n+1} exceeds q, i.e. when its rank is above $\lceil (n+1)(1-\alpha) \rceil$. The number of such ranks is at most $\alpha(n+1)$, so

$$P(\text{miscoverage}) \leq \alpha(n+1)/(n+1) = \alpha \implies P(\text{coverage}) \geq 1 - \alpha$$

■ No distributional assumption appears anywhere: the guarantee is purely combinatorial, which is why the (n+1) correction is not a fudge but the exact count of available ranks.

Worked example

With n = 99 calibration residuals and $\alpha = 0.1$:

$$\lceil (99+1)(1-0.1) \rceil = \lceil 90 \rceil = 90$$

so q is the 90th smallest residual. Nine of the 99 calibration points lie above it, and the guarantee says a fresh point exceeds it with probability at most 10%.

```python
def split_conformal(residuals: np.ndarray, alpha: float = 0.1) -> float:
    """Returns the conformal half-width for 1 - alpha coverage.

    Args:
        residuals: Absolute errors on the calibration set.
        alpha: Miscoverage rate; 0.1 targets 90% coverage.

    Returns:
        The half-width q to add and subtract from a point prediction.

    Raises:
        ValueError: If the calibration set is too small for this alpha.
    """
    s = np.sort(np.asarray(residuals, dtype=np.float64))
    n = s.size
    if n < int(np.ceil(1.0 / alpha)) - 1:
        raise ValueError(f'need at least {int(np.ceil(1/alpha)) - 1} '
                         f'calibration points for alpha={alpha}')
    k = int(np.ceil((n + 1) * (1.0 - alpha))) - 1
    return float(s[min(k, n - 1)])


def coverage(y_true, y_lower, y_upper) -> float:
    """Fraction of truths that fall inside their interval."""
    y_true = np.asarray(y_true)
    return float(np.mean((y_true >= y_lower) & (y_true <= y_upper)))
```

Note the minimum calibration size the code enforces: with $\alpha = 0.05$ you need at least 19 points, because $\lceil (n+1) \cdot 0.95 \rceil$ must not exceed n. Ask for 99% coverage from 50 calibration points and the quantile does not exist — the honest answer is an infinite interval, so the function refuses.

25.3 Adaptive intervals

Plain split conformal gives a **constant** half-width q: the same interval everywhere, regardless of whether a point is in a well-understood region or an extrapolation. Often that is not what you want.

Normalised scores fix it. Use $s_i = |y_i - \hat{y}_i| / \hat{u}_i$ where $\hat{u}$ is a second model predicting the local error magnitude. The interval becomes $\hat{y} \pm q \cdot \hat{u}(x)$ — wider where the model expects to struggle — and the coverage proof is unchanged, because the rank argument never looked at how the score was defined.

Conformalised quantile regression goes further: fit models for the $\alpha/2$ and $1-\alpha/2$ quantiles directly, then conformalise the residuals of those bounds.

25.4 The honest caveats

Exchangeability fails under drift. Time series are the canonical violation: tomorrow is not exchangeable with last year. The guarantee degrades exactly when the process changes — which is when

you needed it. So measure empirical coverage on a held-out *future* window, not only in-sample, and report it.

The literature's upper bound may not hold *[TSP p.332]*. Carry that warning into your own reports rather than quoting a guarantee you have not tested.

Coverage without width is meaningless. An interval from $-\infty$ to $+\infty$ has perfect coverage. Always report both, plus a proper scoring rule such as pinball loss or the Winkler score *[TSP p.346]*. The Winkler score for an interval [l, u] is

$$(u - l) + (2/\alpha)\cdot(l - y) \text{ if } y < l, \quad (u - l) + (2/\alpha)\cdot(y - u) \text{ if } y > u, \quad \text{else } (u - l)$$

— width, plus a penalty proportional to how far outside the truth fell. It is minimised by an interval that is both narrow *and* correctly covering, which is exactly the trade you want scored.

For time series, **EnbPI** *[TSP p.337]* replaces the single calibration split with bootstrap ensembles:

1. Draw B bootstrap samples of the training series; fit a model on each.
2. For each training point, aggregate the predictions of only those models that did **not** see it — a leave-one-out residual at no extra cost.
3. Form intervals from the quantiles of those residuals, sliding the residual window forward as new observations arrive.

This keeps more data in training and adapts to drift. *[TSP p.340]* applies it to a PyTorch model directly.

25.5 Anomaly detection, briefly

Conformal intervals give anomaly detection for free: a point outside its interval is non-conforming at level α *[TSP ch.17 p.690]*. Other approaches — isolation forests, autoencoder reconstruction error, Matrix Profile — differ mainly in what "normal" means.

The hard part is never the algorithm; it is evaluation. Labels are scarce, subjective and usually retrospective. Report precision@k against whatever labels you have, say exactly how they were produced, and resist any claim that rests on a plot looking right.

Part IV gate

Check	Threshold
Your ACF of a simulated AR(0.7)	$\rho(k) \approx 0.7^k$ within sampling error
Your ACF/PACF vs `statsmodels`	1e-8
ADF, KPSS and PP all run	both outcomes recorded when they disagree
Fold boundaries match the worked table	exact
MASE worked example	0.625 exactly
MAPE under a unit change	changes 15-fold; MASE unchanged
Neural model vs seasonal naive	reported either way, including a loss
Conformal coverage at $\alpha = 0.2, 0.1, 0.05$	within 2% of target, width reported
Coverage under a deliberate shift	measured, not assumed

Part V — Professional Practice

A correct model that nobody can rerun, read, or trust is not a professional deliverable. This part is about the difference between code that works once and work that a team can own.

26 · The code contract

Every listing in this book follows the Google Python Style Guide. The rules that matter most in machine-learning code, with their anchors:

Rule	Anchor	Why it matters here
80-column lines	*[GPSG p.20 §3.2]*	side-by-side diffs and review
4 spaces, never tabs	*[GPSG p.23 §3.4]*	—
Docstrings on every public module, class and function	*[GPSG p.27 §3.8]*	—
Args / Returns / Raises in function docstrings	*[GPSG p.29 §3.8.3]*	the shape of your data belongs here
Type annotations	*[GPSG p.19 §2.21, p.46 §3.19]*	catches `ndarray` vs `Tensor` before runtime
`module_name`, `ClassName`, `function_name`, `GLOBAL_CONSTANT`	*[GPSG p.43 §3.16]*	—
`main()` plus `if __name__ == '__main__':`	*[GPSG p.45 §3.17]*	makes every script importable and testable
Rethink functions past ~40 lines	*[GPSG p.46 §3.18]*	training loops bloat silently
No bare `except:`	*[GPSG p.5 §2.4]*	a swallowed exception becomes a silent NaN
`TODO(username):`	*[GPSG p.39 §3.12]*	—

One documented exemption: inside a derivation, short mathematical names (`A`, `x`, `lr`, `mu`) matching the source notation are allowed and preferred *[GPSG §3.16.5]*. Local only — never a public API name.

```python
def rmse(y_true: np.ndarray, y_pred: np.ndarray) -> float:
    """Computes the root mean squared error.

    Args:
        y_true: Ground-truth values, shape (n,).
        y_pred: Predicted values, shape (n,).

    Returns:
        The RMSE as a float.

    Raises:
        ValueError: If the shapes do not match.
    """
```

That docstring is not decoration. It states the shapes — the single most common source of silent bugs in array code — and it names the failure mode.

26.1 Conventions the style guide does not cover

The Google guide is about style, not about machine learning. These seven rules are local to this text and carry no external authority; they are stated so you can argue with them rather than absorb them unawares.

1. **One seed per run**, in the config, recorded in the report.
2. **Notebooks explore; modules decide.** A number that reaches a report comes from a module invoked by a script, never from notebook state.
3. **Configs are data.** No experiment reads a value not in its config.
4. **Datasets are addressed by name and checksum**, never by path.
5. **Every run writes an audit pack** — config, report, environment, dataset card, plots.
6. **Failures are results.** A model losing to the baseline is reported with the same prominence as a win.
7. **A number without a tag does not exist.**

27 · The method

27.1 Four roles, never two at once

The accompanying testbed is organised around four roles. You will play all four; the discipline is never to play two simultaneously on the same artifact.

Role	Mandate	Forbidden
Analyst	Extract the mathematics, assumptions and acceptance thresholds from the source.	Writing implementation code.
Coder	Turn that into working, lint-clean, typed code.	Choosing the metric, or editing the threshold.
Auditor	Re-derive the result independently; try to break it.	Fixing what they find — hand it back.
Interface	Build the controls, tables and plots.	Changing numbers to improve a plot.

Sleep between Coder and Auditor on the same artifact. Audit your own code ten minutes after writing it and you reread your intentions, not the code.

27.2 Thresholds before code

The acceptance threshold is written **before** the implementation, by the Analyst, from the source. A threshold adjusted after seeing the result is not a threshold; it is a rationalisation, and it is the most serious defect this method is built to prevent.

27.3 Status tags

Tag	Meaning	Citable?
VERIFIED	Reproduced from a clean run; the numeric gate passed.	yes
PENDING REVISION	A gate failed, or an open bug touches it.	no
UNVERIFIED	Ran, but has no numeric gate — a visual result, a missing dependency, a competency outside the sources.	only with the tag attached
BUG	A defect with a log entry; blocks the gate.	no

Any `UNVERIFIED` or `PENDING` number must carry its tag *everywhere it appears* — report, plot title, table, conversation. A number that travels without its tag is the failure mode the whole structure exists to prevent.

27.4 The audit pack

Every run writes a directory containing the resolved config, the report in both machine and human form, the environment (interpreter, platform, package versions), the dataset card (name, shape, seed, checksum, provenance), and the plots. The test: **a stranger, given only that directory, can rerun the work and match your numbers.** If they cannot, the run is not finished, however good the model is.

28 · From model to system

28.1 Tracking

Log parameters, metrics, artifacts and the environment for every run *[PTG p.363 TensorBoard, p.372 MLflow, p.377 Weights & Biases]*. The gate is blunt: delete a run's local files and rebuild the result from the tracking record alone.

28.2 Serving

Start local *[PTG p.387]*. The things that actually bite in production are rarely the model:

- **Input validation** — reject malformed requests loudly, at the boundary.
- **Training/serving skew** — the same preprocessing code must run in both places. This is the single most common production failure, and shipping the fitted `Pipeline` object rather than reimplementing the transforms is the defence.
- **Latency** — measure p50 *and* p95; the average hides the tail users feel.
- **Rollback** — have one before you need one.
- **Monitoring** — input drift, output drift, and the realised metric once labels arrive.

28.3 Explainability

Tool	Answers	Caveat
Permutation importance	which features the model relies on	compute on **held-out** data, and note it splits credit arbitrarily between correlated features
Partial dependence	the average direction and size of an effect	assumes feature independence
ICE curves	per-sample effects	plot *with* PDP — the average hides heterogeneity
Standardised coefficients	effect size in a linear model	only meaningful after scaling
Local attributions	why *this* prediction	unstable; never present a single run as definitive

Model-agnostic explanations are approximations of the model, not of reality. They describe what the model does, which is not the same as what is true.

28.4 Governance

A model card: intended use, training data and date, metrics with intervals and on which slices, known failure modes, and who to contact. A fairness assessment naming the protected attribute and the harm considered. An incident plan with a rollback trigger. These are not bureaucracy — they are what makes the difference between a model you deployed and a model you are accountable for.

29 · The route through

29.1 Twenty-four weeks

Weeks	Material	You can prove
0	Testbed, code contract, metrics and leakage contracts (Ch 26–27)	the environment boots and the gates run
1–4	Part I (Ch 1–8)	the mathematics, in code you wrote
5–8	Part II (Ch 9–15)	a leak-proof pipeline with honest intervals
9–13	Part III (Ch 16–21)	a network trained from a hand-written loop
14–18	Part IV (Ch 22–25)	a forecast with verified interval coverage
19–22	Part V (Ch 26–28)	a tracked, served, explained model
23–24	Capstone	someone else reproduces every number

Work the matching project in Part VI at the end of each block: project 1 after week 8, project 2 after week 13, projects 3 and 4 after week 18. They are the rehearsal for the capstone, and each one takes under a minute to run.

Compressed to fourteen weeks at double the hours: trim Chapter 3, Chapter 15 and Chapter 19 first. **Never compress** the metric contract, the leakage chapter, temporal evaluation or conformal coverage. Everything else can be relearned later; those four are what make every other number true or false.

29.2 The daily shape

Minutes	Role	Activity
30	Analyst	read the anchored pages; write the threshold down first
60	Coder	implement, test, lint
20	Auditor	run the gate, tag the result, log what broke
10	Interface	update the board, commit the run

29.3 The capstone

One system a team could inherit: data through a single loader with a card and checksum; diagnostics with disagreements recorded; at least four arms including a naive baseline on identical rolling-origin folds; one

scorer; conformal intervals with measured coverage including under a deliberate shift; the leakage suite green; permutation importance and PDP/ICE; a served endpoint with latency numbers; an audit pack; and a written list of everything still `UNVERIFIED`.

It is finished when a person who has not seen your work clones the repository, runs one command, and reproduces every headline number within its stated interval.

Part VI — Four Worked Projects

Everything so far has been pieces. This part is five complete systems, each a single runnable file in `ml-mastery/projects/`, each going from raw data to a tagged result with an audit pack. Run them before you read about them:

```
cd ml-mastery/projects
python run_all.py
```

On the machine this text was written on: **5/5 verified, 39 gates, 90 seconds**, no downloads.

#	Project	Chapters	Runtime
1	Tabular classification, end to end	9–14	~40 s
2	A convolutional classifier, trained by hand	16–19	~10 s
3	Forecasting with real intervals	22–25	~2 s
4	A VAE with the ELBO taken apart	5, 8, 20	~20 s
5	Serving a model, and watching it afterwards	28	~9 s

> **Read this part differently.** The interesting content is not that the projects pass. It is the three places where they *did not*, where the gate turned out to be wrong rather than the code. Those are in §30.4, §31.3 and §33.2, and they are the most honest pages in this book.

30 · Project 1 — Tabular classification

30.1 The data is the lesson

The generator is not a convenience; it is where the difficulty lives. It produces an 18% positive rate, values missing *not* at random (dormant accounts have no session data), an 18-level region column with a long tail, and — the important one — **several rows per customer**.

That last pathology is what makes the project real. Rows are not independent, so a plain `KFold` would put the same customer in train and test, and the model would be scored on its ability to recognise people it has already seen. The defence is a group-aware splitter, and the project asserts the result:

```
splitter = StratifiedGroupKFold(n_splits=5, shuffle=True, random_state=0)
dev_idx, test_idx = next(splitter.split(frame, labels, groups))
overlap = set(groups[dev_idx]) & set(groups[test_idx])
# gate: len(overlap) == 0
```

30.2 The threshold is a business parameter

Chapter 9 derived p* = C_FP/(C_FP + C_FN). With a £5 false alarm and a £120 missed churn, that is 0.04 — not 0.5. The project searches for the cost-minimising threshold on validation data and finds **0.033**, close to theory and different from it because the model's probabilities are not perfect.

The number that matters is what it saves:

Threshold	Test cost
0.500 (the default nobody chose)	£15,055
0.033 (chosen from the costs)	£2,460

Six times the loss, from a single line nobody thought to question. No modelling change produces an improvement that size.

30.3 Nested CV, and reading the gap

The project runs group-aware nested CV across three model families. The optimism gap — flat minus nested — is the selection bias of Chapter 14, made visible. In `--quick` mode it is near zero, which is itself the lesson: with four configurations there is almost nothing to overfit. Widen the grid and watch it grow.

30.4 Where the gate was wrong

The first version compared the nested score against `search.best_score_` from the *inner* loop, and the gate "nested ≤ flat" failed: the nested score came out **higher**.

That is not selection bias running backwards. The inner CV trains on three-quarters of an outer training fold, so its models see less data and score worse for a reason that has nothing to do with optimism. The two numbers were not comparable.

The fix was to measure the flat score on the **same folds**:

```
# Select and report on the identical outer folds -- the mistake the
# nested procedure exists to correct.
flat = GridSearchCV(pipeline, grid, scoring='average_precision',
                    cv=outer_cv, n_jobs=-1, refit=False)
flat.fit(x, y, groups=groups)
```

The code was right and the gate was wrong. That distinction is worth more than the project: a failing check is a claim about your understanding, not only about your code.

---

31 · Project 2 — A convolutional classifier, trained by hand

31.1 Check the gradients before you trust the training

Before a single epoch, the project verifies autograd against central differences on the real model, in double precision with dropout off:

```
worst relative gap autograd vs central difference: 7.31e-09
```

Chapter 16 explained why both details matter: dropout makes the function stochastic so the comparison is meaningless, and float32's $\varepsilon \approx 10^{-7}$ puts the round-off floor above the discrepancy you are hunting.

31.2 Measure the learning rate; do not guess it

The LR range test sweeps geometrically from 1e-4 to 1.0 and records the loss. The minimum lands near 0.73; the project takes an order of magnitude below it, 0.073, and trains. This is the 2/L edge of Chapter 6 appearing as an empirical curve.

Then three checks that most projects skip:

Check	Result
Determinism: a second run at the same seed	identical losses
Checkpoint round-trip: reloaded model's logits	`0.00e+00` difference
Temperature scaling changes accuracy	`0.9694` → `0.9694`, exactly

The third is a *theoretical* gate: dividing logits by a positive constant cannot change their ranking, so accuracy must be unchanged to the last bit. If it moves, the implementation is wrong — and the gate would catch it.

31.3 Where the gate was wrong, again

The calibration gate said "test ECE after ≤ ECE before". It failed: 0.0140 → 0.0145.

The model was already well calibrated. Temperature was fitted on 288 validation points, and on 360 test points the ECE's own sampling noise is about $1/\sqrt{360} \approx 0.05$ — eighty times the movement being treated as a regression. The original gate was testing noise.

The replacement states what is actually claimable:

Gate	Observed
ECE improves on the data T was fitted on	0.0193 → **0.0044**
Test ECE moves less than $1/\sqrt{n} = 0.053$	+0.0006

A calibration method that does not help an already-calibrated model is not a bug. Reporting that honestly is the job.

32 · Project 3 — Forecasting with real intervals

32.1 Diagnostics first

ADF and KPSS are both run and **both outcomes recorded** — the discipline of Chapter 22, enforced in code:

```
ADF p=0.9206  KPSS p=0.0100
reading: both agree: non-stationary
```

Four combinations are possible and the project names all of them; picking whichever test suits your plan is the quiet dishonesty that nothing in the output would otherwise reveal.

32.2 Baselines, and what beating them means

Model	MASE	RMSSE
ridge on lags	**0.744**	0.760
gradient boosting on lags	0.883	0.901
seasonal naive	1.100	1.087
mean	1.989	1.929
naive	2.771	2.687
drift	3.095	2.961

Ridge on lag features beats the best baseline by 32%. Note that the simplest model wins — and note how badly drift does, which tells you the series is not a random walk, consistent with the diagnostics.

Fold boundaries are printed and asserted, so a reviewer can verify the evaluation without reading the code:

```
fold 0: train [0,499] < test [500,527]  (series t=528..555)
fold 1: train [28,527] < test [528,555] (series t=556..583)
```

32.3 The headline: coverage fails exactly where it must

Split conformal at $\alpha = 0.1$, calibrated on held-out training data:

Segment	n	Coverage
pre-shift	410	**0.946**
post-shift	350	**0.734**

Target was 0.90. Before the regime shift, coverage holds. After it, **twenty-one points of coverage are gone**.

Chapter 25 proved the guarantee from exchangeability alone. A permanent level shift destroys exchangeability, so the guarantee evaporates — and it evaporates precisely when you most needed a trustworthy interval, because something changed.

This is the single most important result in the four projects. It is why coverage must be *measured on a future window*, not assumed from the theorem, and why the project reports interval width and Winkler score beside coverage: an infinitely wide interval covers everything.

---

33 · Project 4 — A VAE with the ELBO taken apart

33.1 The beta sweep

β	Reconstruction	KL	Active units	Reading
0.0	22.66	65.36	8/8	no KL penalty: a plain autoencoder
0.5	24.45	1.91	1/8	both terms alive
1.0	26.25	0.45	0/8	posterior collapse
4.0	27.01	0.003	0/8	posterior collapse
20.0	27.02	0.0002	0/8	posterior collapse

Read the summed loss alone and these five models look broadly similar. Read the terms apart and they are **completely different models**: one is an unregularised autoencoder, one is balanced, three have switched their latent off entirely.

That is why `elbo_terms` returns a tuple rather than a loss.

33.2 Where the gate was wrong, a third time

The first version used summed **squared error** for reconstruction, and the β = 1 model collapsed: KL fell to 0.027, zero active units, reconstruction worse than PCA.

The model was not broken. It was correctly minimising a badly scaled objective. Squared error over 64 pixels in [0, 1] is of order 5; the KL is measured in **nats** and wants tens. Against that exchange rate, the cheapest move is to switch the latent off and accept the reconstruction penalty.

The fix is the one Chapter 8 implies: use the **Bernoulli likelihood**, which is both the correct likelihood for intensities in [0, 1] and measured in nats, so the two terms are commensurable.

> **Posterior collapse is often a units bug, not a modelling mystery.** The project keeps `likelihood='gaussian'` available so you can reproduce the failure deliberately.

33.3 The fair baseline

PCA is the MSE-*optimal* linear map at a given rank (Chapter 5). Judging a BCE-trained model by MSE compares two objectives and calls the difference a result, so the comparison autoencoder is trained on squared error too:

Model	Test reconstruction (MSE)
PCA, 8 components	1.564
autoencoder, β = 0, trained on MSE	**1.430**
VAE, β = 1	2.836 (2/8 units active)

Nonlinearity beats the linear optimum, as it should. The VAE is worse — **and that is the correct outcome**, because it is paying reconstruction for a latent you can sample from. The gate says `vae_pays_for_its_prior` rather than pretending the VAE should win.

34 · Project 5 — Serving a model, and watching it afterwards

The project the other four lead to. A model that never leaves the notebook has not been tested against the thing that actually breaks models in production, which is rarely the model.

The service is real: a socket on loopback, a `ThreadingHTTPServer`, real `urllib` requests. Nothing is mocked, so the latencies are measured rather than asserted.

34.1 Ship the pipeline, not the recipe

The headline result is a zero:

```
max |served − in-process| over 50 rows: 0.00e+00
```

Training/serving skew is the most common production failure in applied machine learning: the preprocessing is reimplemented in the serving layer — in another language, by another team, six months later — and the two implementations drift apart silently. Predictions in production stop matching the ones that were validated, and no test catches it because each half is correct on its own.

The defence is structural rather than vigilant. Chapter 11 put every parameterised transform inside one `Pipeline` object; this project ships *that object*, so there is no second implementation to disagree with the first. The gate demands exact equality, not approximate:

```
Gate('no_training_serving_skew',
     'served predictions equal in-process predictions',
     f'{skew:.2e}', skew < 1e-12)
```

34.2 Reject at the boundary

Request	Response
missing `tenure_months`	400 `missing fields: tenure_months`
`monthly_charge: "free"`	400 `monthly_charge must be numeric, got str`
`days_since_login: Infinity`	400 `days_since_login must be finite`
a JSON array instead of an object	400 `body must be a JSON object`
a valid row	200 `p=0.0316 (v2-boosting)`

Four ways to be wrong, four 400s, no 500s and — crucially — no silently imputed prediction. A service that quietly guesses when the input is malformed produces numbers nobody can audit later.

34.3 Latency, and where it actually goes

```
server-side p50 24.13 ms   p95 26.42 ms   p99 28.57 ms
```

Report **p95**, not the mean: the mean is dragged around by the body of the distribution while users experience the tail.

The more useful observation is that **almost none of those 24 ms is the model**. A single-row `predict_proba` through a `ColumnTransformer` spends its time constructing a one-row DataFrame, dispatching through pandas, and running transformers designed for batches. Batching requests moves this by an order of magnitude; optimising the model would move it by nothing.

That is the general lesson of production performance work: **measure before you optimise, or you will optimise the wrong layer** with great diligence.

34.4 Watching the inputs, because the labels are late

Labels arrive weeks after predictions, so the realised metric is the *last* signal you get, not the first. Input monitoring is what you have in the meantime.

The project uses the **population stability index**, which bins the live sample by the training data's quantiles and sums

$$\mathrm{PSI} = \Sigma_{\text{bins}} \left(p_{\text{live}} - p_{\text{ref}}\right) \cdot \ln\left(p_{\text{live}} \,/\, p_{\text{ref}}\right)$$

The conventional readings: below 0.1 nothing material, 0.1–0.25 moderate, above 0.25 significant. Note the form — it is a symmetrised cousin of the KL divergence of Chapter 8, and non-negative for the same reason.

Feature	In-distribution	After a deliberate shift
support_tickets	0.0037	**7.2187**
days_since_login	0.0177	**6.6652**
tenure_months	0.0207	**1.1913**
avg_session_minutes	0.0169	0.0169
monthly_charge	0.0111	0.0111

Zero false alarms on in-distribution traffic, three features flagged after the shift, and the two untouched features correctly stay silent. A drift detector that fires constantly is worse than none, because it teaches the on-call engineer to ignore it.

The shift here is **covariate shift**: a marketing push brings in newer customers with more tickets and longer gaps between logins. The relationship between features and churn is unchanged — only the input

distribution moved. Output monitoring alone can miss this; input monitoring cannot.

The output moved too, and the project measures that as well: mean score 0.207 → 0.457, output PSI 1.16.

34.5 Rollback is a feature you test

```
before: v2-boosting p=0.0316
after : v1-logistic p=0.2565
matches v1 computed directly: True
```

A registry with two versions and a promote operation, exercised while the service is running. The gate requires the rolled-back service to reproduce v1's probability **exactly**, because a rollback that returns "roughly the old model" is not a rollback.

Have one before you need one. The moment you need it is never the moment to write it.

34.6 What this project does not cover

Honest limits, consistent with Appendix A. This is a single-process service on loopback: no containers, no orchestration, no authentication, no horizontal scaling, no CI/CD, no feature store, and no persistence of the monitoring state between runs. *[PTG p.385–407]* covers local, Heroku and Azure deployment and stops short of all of it too.

What the project *does* establish is the part that is usually skipped and is not infrastructure at all: that the thing you serve computes what you validated, that it refuses bad input, that you know its tail latency, that you would notice the inputs changing, and that you can go back.

---

35 · What the five have in common

1. **Gates declared before the run**, with the rule written out, so a reader can check the threshold against the chapter it came from.
2. **An audit pack** per project in `runs/<name>/report.json`.
3. **Honest failure.** Three of the five shipped with a failing gate on the first run. In all three the gate was wrong, and the correction is documented rather than quietly edited away.
4. **No downloads, no hidden state.** Data generated or bundled, seeds explicit, reruns reproduce.

The habit to take from this part is the third one. A failing check is a claim about your understanding, and the useful question is never "how do I make this pass?" but **"which of us is wrong — the code, or the threshold I wrote down before I understood the problem?"**

Both answers are common. Only one of them is a bug.

Appendix A · What these sources do not cover

Stating the edge of your competence is part of having it. Four areas are not covered by the sources behind this text, and nothing here should be read as teaching them:

Gap	What is missing	What would close it
Reinforcement learning	No RL theory in any of the four books. PPO and RLHF appear only as implementations in `papers/29_PPO` and `papers/30_RLHF`.	Sutton & Barto; the original PPO and RLHF papers.
Distributed training	No DDP, FSDP or ZeRO. The nearest material is PyTorch Lightning *[PTG p.351]* and GPipe in `papers/24_gpipe`.	A dedicated distributed-training source and a multi-GPU machine.
Statistical learning theory	No PAC learning, VC dimension, Rademacher complexity or concentration inequalities. *[MML]* stops at the law of large numbers *[p.595]* and information theory *[p.606]*.	Shalev-Shwartz & Ben-David, or Mohri.
Production engineering	*[PTG p.385–407]* covers local, Heroku and Azure. No containers, CI/CD, feature stores, or registries beyond MLflow. Project 5 covers serving, validation, latency, drift and rollback, but none of the infrastructure around them.	A dedicated MLOps source.

Two further limitations of this text itself. The scikit-learn source is an EPUB with no stable pagination, so its anchors are chapter and section headings rather than pages. The *Time Series with PyTorch* anchors are PDF page indices of the supplied file, not printed-book pages.

Appendix B · The exercises

Every gate in this book is implemented in `exercises.py`, which sits beside this PDF. It is one file, it runs on NumPy alone, and it reports each exercise as `PASS`, `FAIL`, or `TODO` — the last meaning you have not implemented it yet, which is never the same as passing.

```
python exercises.py                 # run everything
python exercises.py --part 1        # one part
python exercises.py --exercise 5.2  # one exercise
python exercises.py --list          # the index
python exercises.py --solutions     # run the reference implementations
```

Work in order. A skipped `TODO` is not a pass, and the summary at the end of the run will tell you so in exactly those terms.

55 exercises in total, every one with a numeric gate taken from a chapter of this text:

Part	Exercises	Covers
1	1.1 – 8.5 (27)	norms, Gram–Schmidt, LU, conditioning, eigenpairs, SVD, PCA, gradient descent and its stability limit, Jacobians, backprop, entropy, KL, MLE
2	9.1 – 15.1 (12)	cost-based thresholds, the leakage demonstration, leak-safe scaling, ridge, the logistic gradient, soft-thresholding, Gini splits, precision/recall, bootstrap intervals, k-means
3	17.1 – 21.1 (6)	stable softmax, the p – y gradient, convolution by im2col and its arithmetic, attention, a PyTorch training step
4	22.1 – 25.2 (10)	autocorrelation, decomposition, rolling-origin folds, MASE and its invariance, APE at zeros, conformal coverage, the Winkler score

Several gates reproduce a worked example from the text exactly — the Gini gains of 0.18 and 0.056, the MASE of 0.625, the eigenvalues 5 and 2 — so when one fails you can find the arithmetic on the page and compare line by line.

Appendix C · The worked projects

Five complete systems in `ml-mastery/projects/`, described in Part VI.

```
cd ml-mastery/projects
python run_all.py              # all four, with a gate summary
python p3_forecast_conformal.py --no-shift    # the stationary control
```

File	Chapters	Gates	Runtime
`p1_tabular_churn.py`	9–14	6	~40 s
`p2_digits_cnn.py`	16–19	8	~10 s
`p3_forecast_conformal.py`	22–25	7	~2 s
`p4_vae_elbo.py`	5, 8, 20	10	~20 s
`p5_serving_monitoring.py`	28	8	~9 s

Each writes an audit pack to `projects/runs/<name>/report.json`. None downloads anything: data is generated or comes from scikit-learn's bundled digits, and Project 5's server binds only to loopback.

Appendix D · Working the exercises in Colab

The 55 exercises exist in two forms. `exercises.py` runs them all from a terminal; four notebooks in `ml-mastery/notebooks/` run the same gates in a browser. The notebooks are **generated from `exercises.py`**, so the thresholds cannot drift apart — there is one source of truth and the notebooks follow it.

Notebook	Part	Exercises
`part1_mathematical_engine.ipynb`	I	28
`part2_learning_from_data.ipynb`	II	12
`part3_neural_networks.ipynb`	III	6
`part4_sequences_uncertainty.ipynb`	IV	9

They are self-contained: no clone, no data upload, no `pip install`. Colab ships NumPy and PyTorch, and nothing is read from disk.

```
1. https://colab.research.google.com
2. File -> Upload notebook, choose one of the four
3. Run the Setup cell once, then work down the page
```

`python exercises.py --colab` prints the same instructions, and `--list` now names the notebook for each part.

If you push this repository to GitHub you can open them without the upload step, at `colab.research.google.com/github/<user>/<repo>/blob/main/ml-mastery/notebooks/<file>.ipynb`. No badge is baked into the notebooks, because a link to a repository that does not exist is worse than no link at all.

The shape of a notebook

Each exercise is three cells: the task and its page anchors, a stub raising `NotImplementedYet`, and a `check('5.2')` call that runs the gate. `check_all()` summarises at the bottom. The Setup cell is collapsed because it carries the reference solutions — two gates compare against them, so they must be in the runtime.

Why the notebooks are tested

A notebook that has never been executed is a liability, so `validate_notebooks.py` runs every cell of every notebook twice:

- **as shipped** — every gate must report `TODO`, never `FAIL`. A stub that *fails* is broken rather than merely unfilled, and the two are not the same condition.
- **as completed** — the reference implementations are bound to the names the exercises expect, and every gate must then `PASS`. Without this pass a notebook could ship a gate nobody could ever satisfy.

Both passes are green for all four notebooks. Writing that validator found a real defect, too: the first generated notebooks flattened each stub's indentation onto its `def` line and would have greeted every reader with an `IndentationError`. Nobody would have noticed by looking.

The projects, too

The five projects of Part VI have notebooks as well, generated from the scripts in `ml-mastery/projects/`:

Notebook	Project	Chapters
`project1_tabular_churn.ipynb`	Tabular classification	9–14
`project2_digits_cnn.ipynb`	A CNN trained by hand	16–19
`project3_forecast_conformal.ipynb`	Forecasting with intervals	22–25
`project4_vae_elbo.ipynb`	A VAE, ELBO taken apart	5, 8, 20
`project5_serving_monitoring.ipynb`	Serving and monitoring	28

Each has the same three-part shape: a **Setup** cell that defines the whole project and *runs nothing*; a cell that runs it end to end with its gates; and three exploration sections that take the pieces apart and give you something to change. Project 5 embeds Project 1's source, since it builds on it.

`validate_project_notebooks.py --run-all` executes every one of them. All five reach `VERIFIED` inside a notebook namespace.

Two bugs the validators caught

Both were invisible to reading, and both would have met every reader on the first cell.

The first generated exercise notebooks used `inspect.cleandoc` on each stub, which dedents the first line and the remainder by *their own* margin — flattening every function body onto its `def` line. Every notebook would have opened with an `IndentationError`.

The project notebooks then raised `NameError: __file__`. A notebook cell has no `__file__`, and the projects use it to locate their `runs/` directory. The generator now defines it against the working directory.

A third, subtler one is worth recording because it was *not* a bug in the notebooks. Validating with a bare `dict` as the namespace made `dataclasses` fail on a string annotation, because it resolves those through `sys.modules[cls.__module__].__dict__` and a dict has no module. Jupyter and Colab register a real

module; the validator now does too. **The harness was unfaithful, not the artifact** — the same distinction as §30.4 and §31.3, one layer further out.

End of text. The gates are the point; go and fail a few.